

Department of Computer Science and Engineering

Assignment Questions

Particular	Details
Course Code & Name	CSA0312 – Data Structures – Slot B
Assignment Title	Design Intelligent Emergency Evacuation Routing System for a city containing more than 8000 intersections and 20,000 weighted roads
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data type for construction of the smart city emergency system. CO4: Make use of heaps, priority queue and hashing techniques for graph storage and route cache for repeated queries in C. CO5: Develop robust smart city emergency system by implementing shortest path algorithms such as Dijkstra's algorithm, Floyd-Warshall.
Bloom's Taxonomy Level	L4 – Analyze
SDG Mapping (SDG 1-17)	SDG 9 - Industry, Innovation, and Infrastructure SDG 11 – Sustainable Cities and Communities SDG 13 – Climate Action

Problem Statement:

A smart-city disaster management authority is developing an Intelligent Emergency Evacuation Routing System for a city containing more than 8000 intersections and 20,000 weighted roads. During floods, earthquakes, industrial accidents, or large-scale fires, road travel costs change dynamically because of congestion, road blockages, damaged infrastructure, and emergency restrictions.

The system receives continuous updates containing intersections, roads, travel costs, road status, evacuation centers, and emergency vehicle locations. It must identify the safest and shortest available route from multiple disaster locations to designated evacuation centers while operating under the following constraints:

- The network contains 8000+ vertices and 20,000+ weighted edges.
- Road weights change frequently because of disaster conditions.
- Blocked roads must be removed or temporarily disabled.
- The system should support multiple source-to-destination queries.
- Repeated evacuation queries should be answered efficiently.
- Memory consumption should remain within moderate computational resources.
- The system should avoid repeatedly processing the same intersections.

- The routing system should prioritize routes with minimum travel cost.
- The system should identify unreachable evacuation centers.
- The implementation must be developed in C.

The engineering team is considering Adjacency Matrix, HashMap-based Adjacency List, HashSet, Min-Heap, Priority Queue, Dijkstra's Algorithm, Floyd-Warshall, and route caching.

Students must:

1. Analyze and justify the choice of graph ADT and graph representation.
2. Design suitable C structures for vertices, edges, hash entries, heap nodes, and routes.
3. Implement dynamic road-weight and road-status updates.
4. Use a Min-Heap to optimize Dijkstra's algorithm.
5. Use a HashSet to track processed intersections.
6. Implement route reconstruction.
7. Implement route caching for repeated queries.
8. Handle blocked roads, invalid vertices, duplicate roads, and unreachable destinations.
9. Compare Dijkstra's algorithm with Floyd-Warshall for the given network.
10. Analyze the time and space complexity of the complete system.
11. Evaluate whether the design can satisfy a 200 ms routing requirement.
12. Analyze how the system behaves when 30% of the roads become unavailable during a disaster.

ANSWER

Student Name	Register Number	Team Members	Date
Athisaya U	192571001	MONIKA B (192571002) PAPAGARI PUSHPA CHARITHA (192525371)	31/08/2026

GitHub Repository: <https://github.com/athisayaprema2007-hue/CSA0312-Intelligent-Evacuation-Routing-System>

A. Student Work

1. Problem Understanding and Formulation

What is the problem?

The city road network must support emergency evacuation routing while road conditions are changing. Intersections are vertices and bidirectional roads are weighted edges. During disasters, road weights may increase because of congestion or damage, roads may be blocked or reopened, and multiple disaster locations may request routes to evacuation centres. The system therefore needs to find the minimum-cost available route on a large, sparse and dynamically changing graph while reporting invalid or unreachable destinations safely.

Expected Outcomes

- Correct minimum-cost routes for valid source-to-destination queries.
- Dynamic handling of road-weight changes, blocking and reopening.
- Efficient repeated-query handling through version-aware route caching.
- Explicit detection of invalid vertices, duplicate roads, negative weights and unreachable destinations.
- Performance comfortably below the 200 ms routing requirement on the verified test configuration.
- Graceful behaviour when 30% of roads become unavailable.

Available Information / Data

The assignment specifies a road network of more than 8,000 vertices and 20,000 weighted edges, continuous road-cost/status updates, multiple route queries and a C implementation. For verification, the program generates a deterministic synthetic city with 8,200 intersections and 21,000 logical bidirectional roads (42,000 directed edge records), using seed 20260831. Functional tests use a small hand-verifiable graph and the full-scale benchmark uses ten fixed source-to-destination queries.

Assumptions

- Road weights are non-negative so Dijkstra's algorithm is valid.
- Each logical city road is bidirectional and is represented internally by two directed edge records.
- Blocked roads remain stored but are marked inactive so they can later be reopened without data loss.
- The full-scale city is deterministic synthetic data for course evaluation; no real-world deployment or live GPS/traffic validation is claimed.
- One scalar travel cost represents the combined routing cost used by the prototype.

Constraints to be Satisfied

- 8,000+ vertices and 20,000+ weighted edges.
- Frequent road-weight and road-status updates.
- Multiple and repeated source-to-destination queries.
- Moderate memory use suitable for a sparse graph.
- No repeated finalization of processed intersections in cyclic graphs.
- Minimum travel cost, unreachable-centre detection and robust validation.
- Implementation in C and evaluation against the 200 ms target.
- Analysis of behaviour after 30% of roads become unavailable.

2. Application of Course Knowledge

The solution applies graph ADTs, hashing, heaps, priority queues and shortest-path algorithms as an integrated data-structures design. The graph is sparse, so a HashMap-based adjacency list is selected instead of an adjacency matrix. A binary Min-Heap implements the Dijkstra priority queue, a HashSet tracks finalized intersections, predecessor arrays support route reconstruction, and a hash-based route cache accelerates repeated queries.

Graph Representation Selection

Representation	Time / Space Effect	Decision
Adjacency Matrix	$O(V^2)$ memory; $O(V)$ neighbour scan	Rejected
HashMap Adjacency List	$O(V+E)$ memory; $O(\text{degree})$ neighbour scan	Selected
Floyd-Warshall Matrix	$O(V^3)$ preprocessing and $O(V^2)$ memory	Rejected at full scale

With 8,200 intersections, the number of possible undirected pairs exceeds 33 million, while the verified network contains only 21,000 logical roads. The adjacency list therefore avoids large matrix waste and lets Dijkstra scan only existing neighbours.

C Structure Design

Structure	Important Fields	Purpose
Edge	dest_id, weight, active, next	One directed half of a road
Vertex	id, index, edges, degree	Intersection and adjacency-list head
Graph	vertices, by_index, counts, version	Complete mutable road network
HashMapEntry	key, value, next	ID-to-Vertex lookup

Structure	Important Fields	Purpose
HashSetEntry	key, next	Processed-intersection membership
HeapNode / MinHeap	vertex_index, dist, nodes, pos	Priority queue and decrease-key
Route	IDs, length, total_cost, reachable	Reconstructed query result
RouteCacheEntry	src, dst, graph_version, route	Versioned repeated-query result

```

typedef struct Edge {
    int dest_id;
    long weight;
    int active;
    struct Edge *next;
} Edge;

typedef struct Route {
    int src_id, dst_id;
    int *intersection_ids;
    int num_intersections;
    long total_cost;
    int reachable;
} Route;

```

Key Algorithmic Reasoning

The Min-Heap provides $O(\log V)$ insert, extract-min and decrease-key operations. A reverse `pos[vertex_index]` array locates heap nodes in $O(1)$ before the logarithmic heap repair. The HashSet keeps average add/contains operations $O(1)$ and guarantees that cyclic graphs do not repeatedly finalize the same vertex. With an adjacency list and binary heap, an uncached Dijkstra query runs in $O((V+E) \log V)$ time with $O(V)$ working memory in addition to $O(V+E)$ graph storage.

3. Solution / Design / Methodology

The implementation is modular. Road updates mutate the graph while preserving consistency in both directions. Every routing-relevant mutation increments `Graph.version`. Dijkstra ignores inactive edges, the HashSet finalizes each intersection once, the predecessor array reconstructs the route, and the route cache accepts a cached answer only when its graph version matches the current graph.

System Architecture Diagram

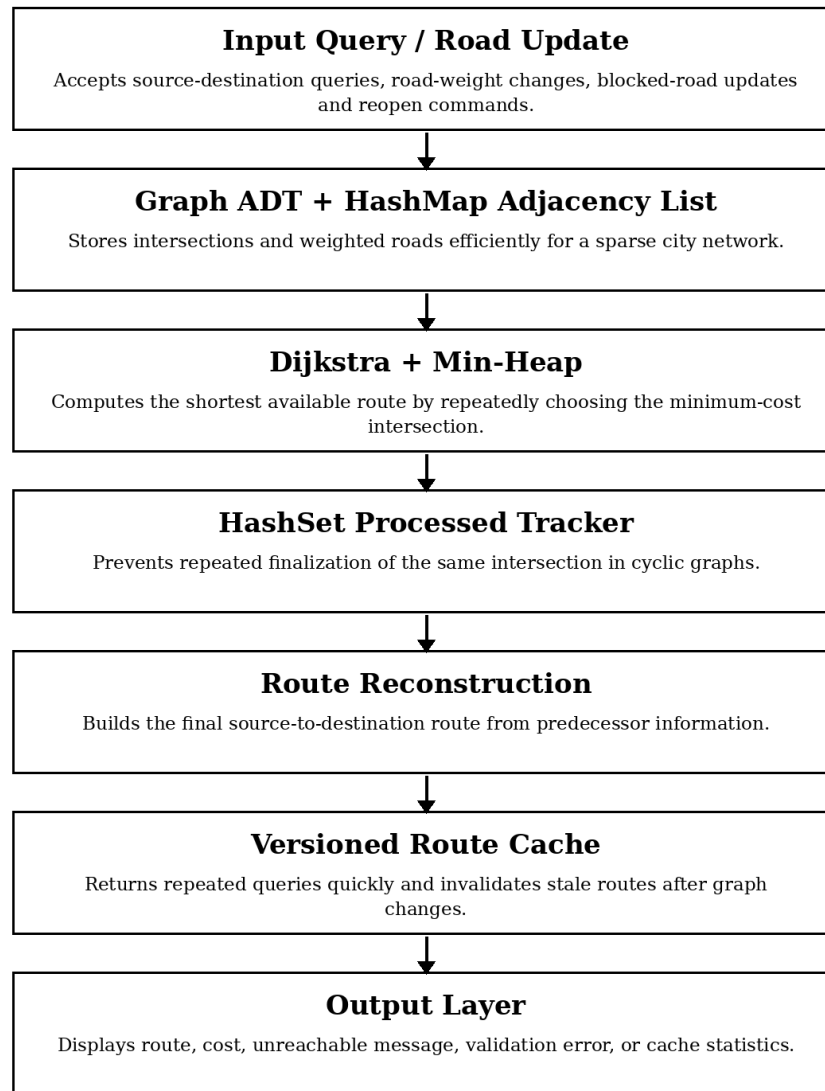


Figure 1. System architecture of the Intelligent Emergency Evacuation Routing System.

Dynamic Road-Weight and Road-Status Updates

Each logical road is stored as two directed Edge records. `graph_update_road_weight` validates both endpoints, rejects negative values, locates both edge records and updates both weights together. `graph_set_road_status` changes both active flags. A blocked road remains stored with `active = 0` so reopening is symmetric. Every routing-relevant mutation increments `Graph.version` because a previously cached route may now be stale.

```

UpdateRoadWeight(G, u, v, w):
    if u or v is invalid: return INVALID_VERTEX
    if w < 0: return NEGATIVE_WEIGHT
    locate edge u->v and edge v->u
    if either edge is absent: return NO_SUCH_ROAD
    update both weights
    G.version = G.version + 1
    return OK

SetRoadStatus(G, u, v, active):
    validate endpoints and locate both edge records
    set both active flags
    G.version = G.version + 1
    return OK

```

Min-Heap Integration

Insert appends a node and sifts upward. Extract-min removes the root, moves the last node to the root and heapifies downward. Decrease-key lowers a tentative distance and sifts upward. Each heap swap updates the reverse position map.

```

ExtractMin(H):
    minimum = H.nodes[0]
    move H.nodes[H.size - 1] to the root
    reduce H.size and repair pos[]
    Heapify(H, 0)
    return minimum

DecreaseKey(H, vertex, new_distance):
    i = H.pos[vertex]
    reject an absent vertex or a non-decreasing value
    H.nodes[i].dist = new_distance
    SiftUp(H, i)

```

Dijkstra and Route Reconstruction

The source and destination are resolved through the graph HashMap. dist[] is initialized to infinity, prev[] to NONE and the source distance to zero. The algorithm repeatedly extracts the minimum tentative distance, finalizes that vertex through the HashSet and relaxes only active outgoing roads. With non-negative weights, the first extraction of the destination proves the shortest distance.

```

Dijkstra(G, src, dst):
    dist[*] = INF; prev[*] = NONE; dist[src] = 0
    insert src into the MinHeap
    processed = empty HashSet
    while heap is not empty:
        u = ExtractMin(heap)
        if u is in processed: continue
        add u to processed
        if u == dst: break
        for every active edge (u, v, w):
            if v is not processed and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                decrease-key or insert v
    return dist and prev

ReconstructRoute(src, dst, dist, prev):
    if dist[dst] == INF: return UNREACHABLE
    count nodes while following prev[] from dst
    allocate route IDs
    fill IDs from the last index down to zero
    return route with total_cost = dist[dst]

```

Versioned Route Cache

RouteCache is a hash table keyed by the ordered pair (source, destination). Each entry stores a deep copy of the Route and the graph version that produced it. A matching version is an average $O(1)$ cache hit. A different version means that the graph changed; the stale entry is removed and the query is recomputed. Unreachable results are cached as well.

```

EvacQuery(G, cache, src, dst):
    validate source and destination
    route = CacheLookup(cache, src, dst, G.version)
    if route exists: return route
    route = Dijkstra plus route reconstruction
    CacheStore(cache, route, G.version)
    return route

```

Exceptional and Failure Handling

Condition	Required Behaviour	Verification
Invalid source or destination	Return NULL / INVALID before search	Tests 6 and 7
Duplicate road	Reject without changing the graph	Test 8
Negative weight	Return GRAPH_ERR_NEGATIVE_WEIGHT	Test 9
Blocked road	Keep the edge; skip active = 0	Test 4
Reopened road	Restore traversal using stored weight	Test 5
Missing road	Return GRAPH_ERR_NO_SUCH_ROAD	Graph module
Unreachable destination	reachable = 0 and cost = -1	Tests 10 and 16
Cyclic graph	HashSet finalizes each vertex once	Test 15

Every allocation is checked. Cleanup functions release graphs, maps, sets, heaps, routes and cache entries. Partial road insertion is rolled back if the second directed edge cannot be allocated.

4. Use of Modern Tools

- GCC compiler for building the modular C implementation with optimization and warning checks.
- Windows Command Prompt for running the guided demo, functional tests, benchmark and road-failure experiment.
- GitHub for version control and submission hosting.
- Microsoft Word and PDF export for preparing the technical report.
- Deterministic result files in results/ (test_results.txt, benchmark_results.csv and road_failure_analysis.txt) so reported values can be independently re-checked from the submitted project.

Tool usage is evidenced by the Windows terminal screenshots below, the generated result files, and the public GitHub repository containing the modular implementation and report.

5. Results and Validation

Functional Test Results

Test Group	Coverage	Result
Shortest route	Hand-verified path and cost	PASS
Dynamic graph	Weight update, block and reopen	PASS
Validation	Invalid IDs, duplicates and negative weights	PASS
Reachability	Disconnected and isolated vertices	PASS
Repeated queries	Cache miss, hit and invalidation	PASS
Cycles	No repeated finalization	PASS
Overall	16 functional tests	16 / 16 PASS

200 ms Performance Validation

The deterministic benchmark uses 8,200 intersections and 21,000 logical bidirectional roads, represented internally by 42,000 directed edge records. Ten fixed source-to-destination queries were executed. The measured uncached average was approximately 3.91 ms per query, the worst observed uncached query was 7.000 ms, and cached hits averaged approximately 0.00003 ms. The measured configuration was Windows 11 on an Intel Core Ultra 7 255H system with 31.4 GB usable RAM, using GCC 3.4.2 with -O2 optimisation.

Metric	Measured Result
Intersections	8,200
Logical roads	21,000
Directed edge records	42,000
Average uncached query	Approximately 3.91 ms
Worst uncached query	7.000 ms
Average cached hit	Approximately 0.00003 ms
Functional tests	16 / 16 PASS

The verified configuration therefore satisfies the 200 ms routing requirement with substantial margin. This conclusion applies to the stated machine, compiler, deterministic data and clock resolution and is not claimed as a universal hardware guarantee.

30% Road-Failure Validation

The failure experiment blocks exactly 6,300 of the 21,000 logical roads using a deterministic shuffle. All ten selected source-destination queries remain reachable, but global reachability from the reference intersection falls from 8,200/8,200 to 8,026/8,200. Therefore 174 intersections, approximately 2.1%, become disconnected from the main reachable region.

Observation	Before Failure	After 30% Failure
Open logical roads	21,000	14,700
Globally reachable intersections	8,200	8,026
Isolated from reference source	0	174
Selected destinations reachable	10 / 10	10 / 10
Cached routes invalidated	0	10
Route-cost change	Baseline	+3 to +175

Route lengths usually increase because blocked segments force detours. The graph version changes after blocking, so all ten previously cached routes are recognized as stale and recomputed. The experiment demonstrates graceful degradation: most of the city remains routable while isolated neighbourhoods are explicitly identified.

Verified Execution Evidence

The following screenshots were captured from the Windows terminal while running the verified program. Timing values can vary slightly between runs because of clock resolution, but all uncached queries remain far below the required 200 ms limit.

```
Command Prompt

[8] Multiple queries from different disaster sites.
Query 2 -> 7 [cache MISS, Dijkstra run]: Route: 2 -> 4 -> 5 -> 6 -> 7 (total travel cost 11, 5 intersections)
Query 3 -> 6 [cache MISS, Dijkstra run]: Route: 3 -> 1 -> 2 -> 4 -> 5 -> 6 (total travel cost 15, 6 intersections)
Query 7 -> 1 [cache MISS, Dijkstra run]: Route: 7 -> 6 -> 5 -> 4 -> 2 -> 1 (total travel cost 15, 6 intersections)

[9] Unreachable center: intersection 8 has no roads.
Query 1 -> 8 [cache MISS, Dijkstra run]: Destination 8 is UNREACHABLE from 1 over active roads.

[10] Invalid queries.
Query 99 -> 4 [cache MISS, Dijkstra run]: rejected: invalid intersection ID
Query 1 -> -5 [cache MISS, Dijkstra run]: rejected: invalid intersection ID

[11] Cache statistics: hits=1 misses=8 stale_evictions=3

Demo finished; freeing all memory.
test_runner: 16/16 tests passed

C:\Users\jimin\OneDrive\Desktop\CSA0312 ASSIGNMENT\CSA0312_clean_submission>
```

Figure 2. Windows terminal screenshot showing multiple routes, unreachable and invalid-query handling, cache statistics, and 16/16 functional tests passed.

```

Command Prompt
Generating deterministic city (seed 20260831): 8200 intersections, 21000 logical roads...
Graph ready: 8200 vertices, 21000 logical roads (42000 directed edges).
Query 1: 4546 -> 2343 cost= 161 hops= 9 uncached avg 1.200 ms c
ached avg 0.000000 ms
Query 2: 488 -> 4320 cost= 154 hops= 5 uncached avg 0.867 ms c
ached avg 0.000010 ms
Query 3: 7680 -> 4213 cost= 212 hops= 9 uncached avg 2.333 ms c
ached avg 0.000030 ms
Query 4: 8005 -> 1947 cost= 223 hops= 11 uncached avg 1.800 ms c
ached avg 0.000010 ms
Query 5: 5674 -> 1101 cost= 241 hops= 7 uncached avg 2.533 ms c
ached avg 0.000000 ms
Query 6: 3568 -> 5188 cost= 215 hops= 12 uncached avg 6.400 ms c
ached avg 0.000020 ms
Query 7: 7204 -> 7818 cost= 238 hops= 13 uncached avg 3.333 ms c
ached avg 0.000000 ms
Query 8: 7116 -> 6482 cost= 228 hops= 9 uncached avg 2.733 ms c
ached avg 0.000010 ms
Query 9: 4541 -> 3854 cost= 241 hops= 10 uncached avg 2.133 ms c
ached avg 0.000020 ms
Query 10: 6012 -> 4594 cost= 265 hops= 7 uncached avg 3.467 ms c
ached avg 0.000020 ms
Benchmark written to results\benchmark_results.csv
Cache stats: hits=1000000 misses=10 stale_evictions=0

C:\Users\jimin\OneDrive\Desktop\CSA0312 ASSIGNMENT\CSA0312_clean_submission>

```

Figure 3. Windows terminal screenshot from the full-scale benchmark with 8,200 intersections, 21,000 logical roads and ten timed route queries.

```

Command Prompt
Building the same deterministic city as the benchmark...
Blocked 6300 of 21000 logical roads (30%).
Analysis written to results\road_failure_analysis.txt

C:\Users\jimin\OneDrive\Desktop\CSA0312 ASSIGNMENT\CSA0312_clean_submission>

```

Figure 4. Windows terminal screenshot from the road-failure experiment, where exactly 6,300 of 21,000 logical roads were blocked and the analysis file was written.

Requirement Traceability Table

Each assignment requirement is mapped to the section that addresses it and the evidence that validates it.

Requirement	Where Addressed	Evidence
8000+ vertices and 20000+ weighted roads	Section 5	Figure 3; results/benchmark_results.csv (8,200 / 21,000)

Requirement	Where Addressed	Evidence
Dynamic road-weight updates	Section 3	Test 3 in results/test_results.txt
Blocked and reopened roads	Section 3	Tests 4 and 5; Figure 2 demo evidence
Multiple source-to-destination queries	Sections 3 and 5	Figure 2; ten timed queries in Figure 3
Repeated-query caching	Section 3	Tests 12-14; cache statistics in Figures 2 and 3
Moderate memory via adjacency list	Sections 2 and 6	$O(V+E)$ storage analysis; representation comparison
HashSet tracking of processed intersections	Sections 2 and 3	Test 15 (cyclic graph, no repeated finalization)
Minimum-cost routing	Sections 2 and 3	Tests 1 and 2 (hand-verified route and cost)
Unreachable destination handling	Sections 3 and 5	Tests 10 and 16; Figure 2
Implementation in C	Sections 2 and 3	src/ modules: graph.c, hashmap.c, hashset.c, minheap.c, dijkstra.c, route_cache.c, main.c
Dijkstra versus Floyd-Warshall comparison	Section 6	Comparison table; 5.5×10^{11} operation estimate
200 ms routing evaluation	Section 5	3.91 ms average and 7.000 ms worst uncached query
30% road-failure analysis	Section 5	Figure 4; results/road_failure_analysis.txt (6,300 blocked; 8,026 reachable)

6. Analysis and Engineering Decision

Comparison of Alternatives

Aspect	Dijkstra with Binary Heap	Floyd-Warshall
Problem solved	Single-source / single-pair	All-pairs
Time	$O((V+E) \log V)$ per query	$O(V^3)$
Space	$O(V+E)$ graph + $O(V)$ working	$O(V^2)$ matrix
Sparse graph	Uses only existing edges	Ignores sparsity
Road update	Next query reads updated graph	Precomputed matrix becomes stale
Repeated query	Average $O(1)$ cache hit	$O(1)$ only after costly preprocessing
Decision	Selected	Unsuitable at full scale

At $V = 8,200$, Floyd-Warshall requires approximately 5.5×10^{11} update operations and about 67.2 million distance cells. Important road changes would also invalidate an all-pairs result. For a sparse, dynamic network with individual route requests, Dijkstra with a binary heap is therefore the appropriate engineering choice.

Time and Space Complexity

Operation	Time Complexity	Space Complexity
HashMap / HashSet lookup	$O(1)$ average	$O(V)$ total
Add intersection	$O(1)$ average	$O(1)$ additional

Operation	Time Complexity	Space Complexity
Add / update / block road	$O(\text{degree})$	$O(1)$ additional
Heap insert / extract / decrease-key	$O(\log V)$	$O(V)$ heap total
Uncached Dijkstra	$O((V+E) \log V)$	$O(V)$ working
Route reconstruction	$O(L)$	$O(L)$
Cache lookup / store	$O(1)$ average	$O(C \times \text{average } L)$
Complete graph storage	-	$O(V+E)$

V denotes intersections, E logical roads, L route length and C cached routes. The complete system uses $O(V+E+C \times \text{average } L)$ memory, which is suitable for moderate computational resources.

Advantages, Limitations and Trade-offs

- Adjacency-list storage scales with existing roads rather than all possible vertex pairs, but neighbour updates require searching the local adjacency list.
- The Min-Heap improves Dijkstra frontier operations, while the additional `pos[]` array uses $O(V)$ memory to support efficient decrease-key.
- The HashSet prevents repeated finalization in cyclic graphs at the cost of an additional $O(V)$ membership structure.
- Global graph-version cache invalidation is simple and correct, but it is conservative because a change on one road can invalidate cached routes that would not actually use that road.
- The benchmark uses deterministic synthetic data and a single scalar travel cost, so the prototype does not claim real-world traffic, safety or GPS validation.
- The implementation is single-threaded and `clock()` has roughly millisecond-scale resolution, limiting precision for extremely fast cached operations.

Final Engineering Decision

The final design combines a HashMap-based adjacency list, Min-Heap Dijkstra, HashSet processed tracking, predecessor-based route reconstruction and a version-stamped route cache. This choice is supported by the sparse network structure, the need for frequent dynamic updates, the 16/16 functional test result and the measured few-millisecond uncached routing time on the verified configuration.

7. Broader Considerations

Sustainability and Environment

- SDG 9 - Industry, Innovation and Infrastructure: the design demonstrates resilient infrastructure software that remains responsive as road conditions change.

- SDG 11 - Sustainable Cities and Communities: the system supports disaster-aware evacuation, rerouting and explicit identification of unreachable neighbourhoods.
- SDG 13 - Climate Action: adaptive routing is relevant to disruptions caused by floods, storms, fires and other climate-related emergencies.

Society and Safety

The safety-oriented behaviour of the prototype is to reject invalid inputs, skip blocked roads and report unreachable destinations explicitly rather than returning a misleading route. This is important in an emergency-routing context because a mathematically invalid or stale route should not be presented as usable guidance.

Ethics, Accessibility and Professional Responsibility

The report distinguishes a course prototype from a deployed emergency system. It uses synthetic data and does not claim real-world disaster validation. A future deployment would require validated map data, accessible user interfaces, reliable live updates, fail-safe operation and review by relevant authorities. The current console interface is useful for technical verification but is not an accessibility-complete public interface.

8. Conclusion

The proposed solution models the city as a HashMap-based adjacency list and answers evacuation queries with binary Min-Heap Dijkstra, a HashSet of finalized intersections, predecessor-based route reconstruction and a version-stamped route cache, implemented as modular C.

The major findings are that all 16 functional tests pass; an uncached query on the 8,200-intersection, 21,000-road city averages about 3.91 ms with a worst observed value of 7.000 ms; a cached hit averages about 0.00003 ms; and blocking exactly 6,300 roads still leaves 8,026 of 8,200 intersections reachable while 174 become isolated and route costs rise by +3 to +175.

The assignment requirements are met through dynamic road updates, Min-Heap Dijkstra, HashSet tracking, route reconstruction, versioned caching, validation, complexity analysis, performance measurement and 30% road-failure testing. The main limitations are the single scalar road cost, global cache invalidation, single-threaded execution, clock resolution and synthetic-data evaluation. Possible improvements include A* or bidirectional search, finer-grained cache invalidation, multi-criteria routing, real road-map import and a user-facing accessible interface.

9. Student Reflection

What I learned beyond the classroom

I learned that choosing an ADT is not only a theoretical complexity exercise; it directly changes whether the whole system scales. The sparse adjacency list avoids the memory cost of a matrix, the Min-Heap changes Dijkstra frontier selection from a linear scan to logarithmic operations, the HashSet makes cyclic-graph processing predictable, and graph-versioned caching shows how data-structure correctness must be maintained when the underlying graph changes. Running the tests and benchmark also showed me why an algorithmic claim should be supported by measured output rather than only by Big-O notation.

What I would improve with more time or resources

With additional time, I would import real road-map data, separate travel time from safety or risk into a multi-criteria cost model, compare Dijkstra against A* and bidirectional search, and replace global cache invalidation with dependency-aware invalidation. I would also use a higher-resolution timing method, add concurrent query handling, and build an accessible graphical interface suitable for non-technical users. These changes would make the prototype closer to a realistic smart-city routing service while preserving the core data-structure design.

10. References

1. CSA0312 assignment problem statement and assessment rubric (course handout).
2. C language and standard-library concepts used for structures, pointers, arrays, dynamic memory and file I/O.
3. GCC compiler and Windows Command Prompt used as the build and execution environment.
4. GitHub repository and version control: <https://github.com/athisayaprema2007-hue/CSA0312-Intelligent-Evacuation-Routing-System>
5. No external road dataset was used; the full-scale network is deterministic synthetic data generated by the submitted program.
6. ChatGPT and Claude were used as AI-assisted tools for drafting, organization, formatting and review support. Technical results, test outputs and benchmark values reported in this assignment were verified through execution of the submitted program.

Appendix A. Full Source Code

The following appendix contains the complete modular C implementation submitted with this assignment. Line numbers are shown in the left margin for reference; source code is reproduced as text without modification.

A.1 src/graph.h

```

1  /*
2  * graph.h
3  * City road network as a HashMap-based adjacency list.
4  *
5  * Design decisions (documented in the report as well):
6  * - Roads are BIDIRECTIONAL: one logical road between intersections u and
7  *   v is stored as two directed edges (u->v and v->u) that are always
8  *   updated together. num_logical_roads counts each road once.
9  * - Every road weight is a non-negative "safety-adjusted travel cost"
10 *   that already folds in distance, congestion, damage and emergency
11 *   restrictions. Negative weights are rejected.
12 * - A blocked road is kept in the list with active == 0, so it can be
13 *   reopened later without losing its weight.
14 * - graph->version increases on every mutation that can change routing
15 *   (road added, weight updated, road blocked/reopened). The route cache
16 *   compares this number to detect stale cached routes.
17 */
18 #ifndef GRAPH_H
19 #define GRAPH_H
20
21 #include "hashmap.h"
22
23 /* Result codes for every mutating graph operation. */
24 typedef enum {
25     GRAPH_OK = 0,
26     GRAPH_ERR_ALLOC,
27     GRAPH_ERR_INVALID_VERTEX,
28     GRAPH_ERR_DUPLICATE_VERTEX,
29     GRAPH_ERR_DUPLICATE_ROAD,
30     GRAPH_ERR_NO_SUCH_ROAD,
31     GRAPH_ERR_NEGATIVE_WEIGHT,
32     GRAPH_ERR_SELF_LOOP
33 } GraphResult;
34
35 /* One directed half of a road, stored in the source vertex's list. */
36 typedef struct Edge {
37     int dest_id;           /* ID of the destination intersection */
38     long weight;           /* non-negative safety-adjusted travel cost */
39     int active;            /* 1 = open, 0 = blocked */
40     struct Edge *next;
41 } Edge;
42
43 /* One intersection. */
44 typedef struct Vertex {
45     int id;                /* external intersection ID (any int) */
46     int index;             /* dense 0..n-1 index for algorithm arrays */
47     Edge *edges;           /* head of the adjacency list */
48     int degree;            /* number of outgoing directed edges */
49 } Vertex;
50
51 /* The whole network. */
52 typedef struct Graph {
53     HashMap *vertices;     /* intersection ID -> Vertex */
54     Vertex **by_index;     /* dense index -> Vertex */
55     int num_vertices;
56     int cap_vertices;      /* allocated length of by_index */
57     long num_logical_roads; /* each bidirectional road counted once */
58     unsigned long version; /* bumped on every routing-relevant change */
59 } Graph;
60

```

```

61 Graph *graph_create(void);
62
63 /* Adds an intersection with the given ID. */
64 GraphResult graph_add_intersection(Graph *g, int id);
65
66 /* Adds one bidirectional road (two directed edges). Rejects unknown
67  * intersections, self-loops, negative weights and duplicate roads. */
68 GraphResult graph_add_road(Graph *g, int u_id, int v_id, long weight);
69
70 /* Changes the weight of an existing road in both directions. */
71 GraphResult graph_update_road_weight(Graph *g, int u_id, int v_id,
72                                     long new_weight);
73
74 /* Blocks (active = 0) or reopens (active = 1) an existing road in both
75  * directions. Blocking an already-blocked road is a harmless no-op. */
76 GraphResult graph_set_road_status(Graph *g, int u_id, int v_id, int active);
77
78 /* Returns the vertex for an ID, or NULL if the ID is unknown. */
79 Vertex *graph_get_vertex(const Graph *g, int id);
80
81 /* Returns the directed edge u->v, or NULL if absent. */
82 Edge *graph_find_edge(const Graph *g, int u_id, int v_id);
83
84 /* Human-readable name for a result code. */
85 const char *graph_result_str(GraphResult r);
86
87 /* Frees the graph and everything it owns. */
88 void graph_free(Graph *g);
89
90 #endif /* GRAPH_H */

```

A.2 src/graph.c

```

1  /*
2   * graph.c
3   * HashMap-based adjacency-list graph with dynamic weight and status
4   * updates, validation, and a version counter for cache invalidation.
5   */
6  #include <stdlib.h>
7  #include "graph.h"
8
9  #define GRAPH_INITIAL_CAPACITY 64
10
11 Graph *graph_create(void)
12 {
13     Graph *g = (Graph *)malloc(sizeof(Graph));
14
15     if (g == NULL) {
16         return NULL;
17     }
18     g->vertices = hashmap_create(GRAPH_INITIAL_CAPACITY);
19     g->by_index = (Vertex **)malloc((size_t)GRAPH_INITIAL_CAPACITY
20                                     * sizeof(Vertex *));
21     if (g->vertices == NULL || g->by_index == NULL) {
22         hashmap_free(g->vertices);
23         free(g->by_index);
24         free(g);
25         return NULL;
26     }
27     g->num_vertices = 0;
28     g->cap_vertices = GRAPH_INITIAL_CAPACITY;
29     g->num_logical_roads = 0;
30     g->version = 0;
31     return g;
32 }
33
34 GraphResult graph_add_intersection(Graph *g, int id)
35 {
36     Vertex *v;
37

```

```

38 | if (hashmap_contains(g->vertices, id)) {
39 |     return GRAPH_ERR_DUPLICATE_VERTEX;
40 | }
41 | if (g->num_vertices == g->cap_vertices) {
42 |     int new_cap = g->cap_vertices * 2;
43 |     Vertex **grown = (Vertex **)realloc(g->by_index,
44 |                                         (size_t)new_cap * sizeof(Vertex *));
45 |     if (grown == NULL) {
46 |         return GRAPH_ERR_ALLOC;
47 |     }
48 |     g->by_index = grown;
49 |     g->cap_vertices = new_cap;
50 | }
51 | v = (Vertex *)malloc(sizeof(Vertex));
52 | if (v == NULL) {
53 |     return GRAPH_ERR_ALLOC;
54 | }
55 | v->id = id;
56 | v->index = g->num_vertices;
57 | v->edges = NULL;
58 | v->degree = 0;
59 | if (hashmap_put(g->vertices, id, v) < 0) {
60 |     free(v);
61 |     return GRAPH_ERR_ALLOC;
62 | }
63 | g->by_index[g->num_vertices] = v;
64 | g->num_vertices++;
65 | return GRAPH_OK;
66 | }
67 |
68 | Vertex *graph_get_vertex(const Graph *g, int id)
69 | {
70 |     return (Vertex *)hashmap_get(g->vertices, id);
71 | }
72 |
73 | Edge *graph_find_edge(const Graph *g, int u_id, int v_id)
74 | {
75 |     Vertex *u = graph_get_vertex(g, u_id);
76 |     Edge *e;
77 |
78 |     if (u == NULL) {
79 |         return NULL;
80 |     }
81 |     for (e = u->edges; e != NULL; e = e->next) {
82 |         if (e->dest_id == v_id) {
83 |             return e;
84 |         }
85 |     }
86 |     return NULL;
87 | }
88 |
89 | /* Prepends one directed edge to a vertex's adjacency list. */
90 | static GraphResult add_directed_edge(Vertex *from, int dest_id, long weight)
91 | {
92 |     Edge *e = (Edge *)malloc(sizeof(Edge));
93 |
94 |     if (e == NULL) {
95 |         return GRAPH_ERR_ALLOC;
96 |     }
97 |     e->dest_id = dest_id;
98 |     e->weight = weight;
99 |     e->active = 1;
100 |     e->next = from->edges;
101 |     from->edges = e;
102 |     from->degree++;
103 |     return GRAPH_OK;
104 | }
105 |
106 | GraphResult graph_add_road(Graph *g, int u_id, int v_id, long weight)
107 | {

```

```

108 Vertex *u = graph_get_vertex(g, u_id);
109 Vertex *v = graph_get_vertex(g, v_id);
110 GraphResult r;
111
112 if (u == NULL || v == NULL) {
113     return GRAPH_ERR_INVALID_VERTEX;
114 }
115 if (u_id == v_id) {
116     return GRAPH_ERR_SELF_LOOP;
117 }
118 if (weight < 0) {
119     return GRAPH_ERR_NEGATIVE_WEIGHT;
120 }
121 if (graph_find_edge(g, u_id, v_id) != NULL) {
122     return GRAPH_ERR_DUPLICATE_ROAD;
123 }
124 r = add_directed_edge(u, v_id, weight);
125 if (r != GRAPH_OK) {
126     return r;
127 }
128 r = add_directed_edge(v, u_id, weight);
129 if (r != GRAPH_OK) {
130     /* Roll back the first half so the road is not left one-way. */
131     Edge *half = u->edges;
132     u->edges = half->next;
133     u->degree--;
134     free(half);
135     return r;
136 }
137 g->num_logical_roads++;
138 g->version++;
139 return GRAPH_OK;
140 }
141
142 GraphResult graph_update_road_weight(Graph *g, int u_id, int v_id,
143                                     long new_weight)
144 {
145     Edge *uv;
146     Edge *vu;
147
148     if (graph_get_vertex(g, u_id) == NULL
149         || graph_get_vertex(g, v_id) == NULL) {
150         return GRAPH_ERR_INVALID_VERTEX;
151     }
152     if (new_weight < 0) {
153         return GRAPH_ERR_NEGATIVE_WEIGHT;
154     }
155     uv = graph_find_edge(g, u_id, v_id);
156     vu = graph_find_edge(g, v_id, u_id);
157     if (uv == NULL || vu == NULL) {
158         return GRAPH_ERR_NO_SUCH_ROAD;
159     }
160     uv->weight = new_weight;
161     vu->weight = new_weight;
162     g->version++;
163     return GRAPH_OK;
164 }
165
166 GraphResult graph_set_road_status(Graph *g, int u_id, int v_id, int active)
167 {
168     Edge *uv;
169     Edge *vu;
170
171     if (graph_get_vertex(g, u_id) == NULL
172         || graph_get_vertex(g, v_id) == NULL) {
173         return GRAPH_ERR_INVALID_VERTEX;
174     }
175     uv = graph_find_edge(g, u_id, v_id);
176     vu = graph_find_edge(g, v_id, u_id);
177     if (uv == NULL || vu == NULL) {

```

```

178     return GRAPH_ERR_NO_SUCH_ROAD;
179 }
180 uv->active = active ? 1 : 0;
181 vu->active = uv->active;
182 g->version++;
183 return GRAPH_OK;
184 }
185
186 const char *graph_result_str(GraphResult r)
187 {
188     switch (r) {
189         case GRAPH_OK: return "OK";
190         case GRAPH_ERR_ALLOC: return "allocation failure";
191         case GRAPH_ERR_INVALID_VERTEX: return "invalid intersection";
192         case GRAPH_ERR_DUPLICATE_VERTEX: return "duplicate intersection";
193         case GRAPH_ERR_DUPLICATE_ROAD: return "duplicate road";
194         case GRAPH_ERR_NO_SUCH_ROAD: return "no such road";
195         case GRAPH_ERR_NEGATIVE_WEIGHT: return "negative weight rejected";
196         case GRAPH_ERR_SELF_LOOP: return "self-loop rejected";
197         default: return "unknown";
198     }
199 }
200
201 void graph_free(Graph *g)
202 {
203     int i;
204
205     if (g == NULL) {
206         return;
207     }
208     for (i = 0; i < g->num_vertices; i++) {
209         Vertex *v = g->by_index[i];
210         Edge *e = v->edges;
211         while (e != NULL) {
212             Edge *next = e->next;
213             free(e);
214             e = next;
215         }
216         free(v);
217     }
218     hashmap_free(g->vertices);
219     free(g->by_index);
220     free(g);
221 }

```

A.3 src/hashmap.h

```

1 /*
2  * hashmap.h
3  * Separate-chaining hash map from integer keys to void* values.
4  * Used by the graph to map intersection IDs to Vertex records, so that
5  * vertex lookup by ID is O(1) on average even for arbitrary, sparse IDs.
6  */
7 #ifndef HASHMAP_H
8 #define HASHMAP_H
9
10 /* One key/value pair inside a bucket chain. */
11 typedef struct HashMapEntry {
12     int key;
13     void *value;
14     struct HashMapEntry *next;
15 } HashMapEntry;
16
17 typedef struct HashMap {
18     HashMapEntry **buckets;
19     int num_buckets; /* always a power of two */
20     int size; /* number of stored entries */
21 } HashMap;
22
23 /* Creates a map. initial_buckets is rounded up to a power of two (min 8).

```

```

24  * Returns NULL on allocation failure. */
25  HashMap *hashmap_create(int initial_buckets);
26
27  /* Inserts or replaces. Returns 1 if a new entry was inserted,
28   * 0 if an existing key's value was replaced, -1 on allocation failure. */
29  int hashmap_put(HashMap *map, int key, void *value);
30
31  /* Returns the stored value, or NULL if the key is absent. */
32  void *hashmap_get(const HashMap *map, int key);
33
34  /* Returns 1 if the key is present, 0 otherwise. */
35  int hashmap_contains(const HashMap *map, int key);
36
37  /* Removes a key. Returns 1 if removed, 0 if it was not present.
38   * The stored value itself is not freed (the caller owns it). */
39  int hashmap_remove(HashMap *map, int key);
40
41  /* Frees the map's own memory. Stored values are not freed. */
42  void hashmap_free(HashMap *map);
43
44  #endif /* HASHMAP_H */

```

A.4 src/hashmap.c

```

1  /*
2   * hashmap.c
3   * Separate-chaining hash map (int key -> void* value) with automatic
4   * resizing. Average O(1) put/get/remove.
5   */
6  #include <stdlib.h>
7  #include "hashmap.h"
8
9  #define HASHMAP_MIN_BUCKETS 8
10 #define HASHMAP_MAX_LOAD_NUM 3 /* resize when size > buckets * 3/4 */
11 #define HASHMAP_MAX_LOAD_DEN 4
12
13 /* Knuth multiplicative hash; keys may be any int, including negatives. */
14 static unsigned long hashmap_hash(int key)
15 {
16     return (unsigned long)((unsigned int)key * 2654435761u);
17 }
18
19 static int round_up_pow2(int n)
20 {
21     int p = HASHMAP_MIN_BUCKETS;
22     while (p < n && p < (1 << 30)) {
23         p <= 1;
24     }
25     return p;
26 }
27
28 HashMap *hashmap_create(int initial_buckets)
29 {
30     HashMap *map = (HashMap *)malloc(sizeof(HashMap));
31     if (map == NULL) {
32         return NULL;
33     }
34     map->num_buckets = round_up_pow2(initial_buckets);
35     map->size = 0;
36     map->buckets = (HashMapEntry **)calloc((size_t)map->num_buckets,
37                                           sizeof(HashMapEntry *));
38     if (map->buckets == NULL) {
39         free(map);
40         return NULL;
41     }
42     return map;
43 }
44
45 /* Doubles the bucket array and re-links every entry. */
46 static int hashmap_resize(HashMap *map)

```

```

47 {
48     int new_count = map->num_buckets * 2;
49     HashMapEntry **new_buckets =
50         (HashMapEntry **)calloc((size_t)new_count, sizeof(HashMapEntry *));
51     int i;
52
53     if (new_buckets == NULL) {
54         return -1;
55     }
56     for (i = 0; i < map->num_buckets; i++) {
57         HashMapEntry *entry = map->buckets[i];
58         while (entry != NULL) {
59             HashMapEntry *next = entry->next;
60             unsigned long idx = hashmap_hash(entry->key)
61                 & (unsigned long)(new_count - 1);
62             entry->next = new_buckets[idx];
63             new_buckets[idx] = entry;
64             entry = next;
65         }
66     }
67     free(map->buckets);
68     map->buckets = new_buckets;
69     map->num_buckets = new_count;
70     return 0;
71 }
72
73 int hashmap_put(HashMap *map, int key, void *value)
74 {
75     unsigned long idx;
76     HashMapEntry *entry;
77
78     if (map->size * HASHMAP_MAX_LOAD_DEN
79         > map->num_buckets * HASHMAP_MAX_LOAD_NUM) {
80         if (hashmap_resize(map) != 0) {
81             return -1;
82         }
83     }
84     idx = hashmap_hash(key) & (unsigned long)(map->num_buckets - 1);
85     for (entry = map->buckets[idx]; entry != NULL; entry = entry->next) {
86         if (entry->key == key) {
87             entry->value = value;
88             return 0;
89         }
90     }
91     entry = (HashMapEntry *)malloc(sizeof(HashMapEntry));
92     if (entry == NULL) {
93         return -1;
94     }
95     entry->key = key;
96     entry->value = value;
97     entry->next = map->buckets[idx];
98     map->buckets[idx] = entry;
99     map->size++;
100    return 1;
101 }
102
103 void *hashmap_get(const HashMap *map, int key)
104 {
105     unsigned long idx = hashmap_hash(key) & (unsigned long)(map->num_buckets - 1);
106     const HashMapEntry *entry;
107
108     for (entry = map->buckets[idx]; entry != NULL; entry = entry->next) {
109         if (entry->key == key) {
110             return entry->value;
111         }
112     }
113     return NULL;
114 }
115
116 int hashmap_contains(const HashMap *map, int key)

```

```

117 {
118     unsigned long idx = hashmap_hash(key) & (unsigned long)(map->num_buckets - 1);
119     const HashMapEntry *entry;
120
121     for (entry = map->buckets[idx]; entry != NULL; entry = entry->next) {
122         if (entry->key == key) {
123             return 1;
124         }
125     }
126     return 0;
127 }
128
129 int hashmap_remove(HashMap *map, int key)
130 {
131     unsigned long idx = hashmap_hash(key) & (unsigned long)(map->num_buckets - 1);
132     HashMapEntry *entry = map->buckets[idx];
133     HashMapEntry *prev = NULL;
134
135     while (entry != NULL) {
136         if (entry->key == key) {
137             if (prev == NULL) {
138                 map->buckets[idx] = entry->next;
139             } else {
140                 prev->next = entry->next;
141             }
142             free(entry);
143             map->size--;
144             return 1;
145         }
146         prev = entry;
147         entry = entry->next;
148     }
149     return 0;
150 }
151
152 void hashmap_free(HashMap *map)
153 {
154     int i;
155
156     if (map == NULL) {
157         return;
158     }
159     for (i = 0; i < map->num_buckets; i++) {
160         HashMapEntry *entry = map->buckets[i];
161         while (entry != NULL) {
162             HashMapEntry *next = entry->next;
163             free(entry);
164             entry = next;
165         }
166     }
167     free(map->buckets);
168     free(map);
169 }

```

A.5 src/hashset.h

```

1 /*
2  * hashset.h
3  * Separate-chaining hash set of integers.
4  * Used inside Dijkstra's algorithm to record intersections that have
5  * already been finalized, so no intersection is ever processed twice
6  * (this is what makes cyclic road networks safe to traverse).
7  */
8 #ifndef HASHSET_H
9 #define HASHSET_H
10
11 typedef struct HashSetEntry {
12     int key;
13     struct HashSetEntry *next;
14 } HashSetEntry;

```



```

15
16 typedef struct HashSet {
17     HashSetEntry **buckets;
18     int num_buckets; /* always a power of two */
19     int size; /* number of stored keys */
20 } HashSet;
21
22 /* Creates a set. initial_buckets is rounded up to a power of two (min 8).
23  * Returns NULL on allocation failure. */
24 HashSet *hashset_create(int initial_buckets);
25
26 /* Adds a key. Returns 1 if newly added, 0 if already present,
27  * -1 on allocation failure. */
28 int hashset_add(HashSet *set, int key);
29
30 /* Returns 1 if the key is present, 0 otherwise. */
31 int hashset_contains(const HashSet *set, int key);
32
33 /* Removes every key but keeps the set usable. */
34 void hashset_clear(HashSet *set);
35
36 /* Frees the set. */
37 void hashset_free(HashSet *set);
38
39 #endif /* HASHSET_H */

```

A.6 src/hashset.c

```

1 /*
2  * hashset.c
3  * Separate-chaining integer hash set with automatic resizing.
4  * Average O(1) add/contains.
5  */
6 #include <stdlib.h>
7 #include "hashset.h"
8
9 #define HASHSET_MIN_BUCKETS 8
10 #define HASHSET_MAX_LOAD_NUM 3 /* resize when size > buckets * 3/4 */
11 #define HASHSET_MAX_LOAD_DEN 4
12
13 static unsigned long hashset_hash(int key)
14 {
15     return (unsigned long)((unsigned int)key * 2654435761u);
16 }
17
18 static int round_up_pow2(int n)
19 {
20     int p = HASHSET_MIN_BUCKETS;
21     while (p < n && p < (1 << 30)) {
22         p <= 1;
23     }
24     return p;
25 }
26
27 HashSet *hashset_create(int initial_buckets)
28 {
29     HashSet *set = (HashSet *)malloc(sizeof(HashSet));
30     if (set == NULL) {
31         return NULL;
32     }
33     set->num_buckets = round_up_pow2(initial_buckets);
34     set->size = 0;
35     set->buckets = (HashSetEntry **)calloc((size_t)set->num_buckets,
36                                           sizeof(HashSetEntry *));
37     if (set->buckets == NULL) {
38         free(set);
39         return NULL;
40     }
41     return set;
42 }

```

```

43
44 static int hashset_resize(HashSet *set)
45 {
46     int new_count = set->num_buckets * 2;
47     HashSetEntry **new_buckets =
48         (HashSetEntry **)calloc((size_t)new_count, sizeof(HashSetEntry *));
49     int i;
50
51     if (new_buckets == NULL) {
52         return -1;
53     }
54     for (i = 0; i < set->num_buckets; i++) {
55         HashSetEntry *entry = set->buckets[i];
56         while (entry != NULL) {
57             HashSetEntry *next = entry->next;
58             unsigned long idx = hashset_hash(entry->key)
59                 & (unsigned long)(new_count - 1);
60             entry->next = new_buckets[idx];
61             new_buckets[idx] = entry;
62             entry = next;
63         }
64     }
65     free(set->buckets);
66     set->buckets = new_buckets;
67     set->num_buckets = new_count;
68     return 0;
69 }
70
71 int hashset_add(HashSet *set, int key)
72 {
73     unsigned long idx;
74     HashSetEntry *entry;
75
76     if (set->size * HASHSET_MAX_LOAD_DEN
77         > set->num_buckets * HASHSET_MAX_LOAD_NUM) {
78         if (hashset_resize(set) != 0) {
79             return -1;
80         }
81     }
82     idx = hashset_hash(key) & (unsigned long)(set->num_buckets - 1);
83     for (entry = set->buckets[idx]; entry != NULL; entry = entry->next) {
84         if (entry->key == key) {
85             return 0;
86         }
87     }
88     entry = (HashSetEntry *)malloc(sizeof(HashSetEntry));
89     if (entry == NULL) {
90         return -1;
91     }
92     entry->key = key;
93     entry->next = set->buckets[idx];
94     set->buckets[idx] = entry;
95     set->size++;
96     return 1;
97 }
98
99 int hashset_contains(const HashSet *set, int key)
100 {
101     unsigned long idx = hashset_hash(key) & (unsigned long)(set->num_buckets - 1);
102     const HashSetEntry *entry;
103
104     for (entry = set->buckets[idx]; entry != NULL; entry = entry->next) {
105         if (entry->key == key) {
106             return 1;
107         }
108     }
109     return 0;
110 }
111
112 void hashset_clear(HashSet *set)

```

```

113 {
114     int i;
115
116     for (i = 0; i < set->num_buckets; i++) {
117         HashSetEntry *entry = set->buckets[i];
118         while (entry != NULL) {
119             HashSetEntry *next = entry->next;
120             free(entry);
121             entry = next;
122         }
123         set->buckets[i] = NULL;
124     }
125     set->size = 0;
126 }
127
128 void hashset_free(HashSet *set)
129 {
130     if (set == NULL) {
131         return;
132     }
133     hashset_clear(set);
134     free(set->buckets);
135     free(set);
136 }

```

A.7 src/minheap.h

```

1 /*
2  * minheap.h
3  * Binary min-heap keyed by tentative distance, with a position index so
4  * decrease-key runs in  $O(\log n)$ . This is the priority queue that drives
5  * Dijkstra's algorithm: extract-min always yields the unfinalized
6  * intersection with the smallest tentative travel cost.
7  */
8 #ifndef MINHEAP_H
9 #define MINHEAP_H
10
11 /* One heap element: an intersection (by dense array index) and its
12  * current tentative distance from the source. */
13 typedef struct HeapNode {
14     int vertex_index;
15     long dist;
16 } HeapNode;
17
18 typedef struct MinHeap {
19     HeapNode *nodes; /* array-based complete binary tree */
20     int *pos; /* pos[vertex_index] = slot in nodes[], or -1 */
21     int size;
22     int capacity; /* also the size of the pos[] index */
23 } MinHeap;
24
25 /* Creates a heap able to hold `capacity` vertices with indices in
26  * [0, capacity). Returns NULL on allocation failure. */
27 MinHeap *minheap_create(int capacity);
28
29 /* Returns 1 if the heap holds no elements. */
30 int minheap_is_empty(const MinHeap *heap);
31
32 /* Returns 1 if the vertex is currently in the heap. */
33 int minheap_contains(const MinHeap *heap, int vertex_index);
34
35 /* Inserts a vertex with its tentative distance.  $O(\log n)$ .
36  * Returns 0 on success, -1 if full, already present or out of range. */
37 int minheap_insert(MinHeap *heap, int vertex_index, long dist);
38
39 /* Removes and returns the minimum-distance node.  $O(\log n)$ .
40  * The heap must not be empty (check minheap_is_empty first). */
41 HeapNode minheap_extract_min(MinHeap *heap);
42
43 /* Lowers the distance of a vertex already in the heap and restores heap

```

```

44 * order by sifting up.  $O(\log n)$ . Returns 0 on success, -1 if the vertex
45 * is absent or new_dist is not smaller. */
46 int minheap_decrease_key(MinHeap *heap, int vertex_index, long new_dist);
47
48 /* Classic heapify: sifts nodes[i] down until the min-heap property holds
49 * for the subtree rooted at i. Used by extract-min. */
50 void minheap_heapify(MinHeap *heap, int i);
51
52 /* Frees the heap. */
53 void minheap_free(MinHeap *heap);
54
55 #endif /* MINHEAP_H */

```

A.8 src/minheap.c

```

1 /*
2  * minheap.c
3  * Array-based binary min-heap with position tracking for decrease-key.
4  */
5 #include <stdlib.h>
6 #include "minheap.h"
7
8 MinHeap *minheap_create(int capacity)
9 {
10     MinHeap *heap;
11     int i;
12
13     if (capacity < 1) {
14         capacity = 1;
15     }
16     heap = (MinHeap *)malloc(sizeof(MinHeap));
17     if (heap == NULL) {
18         return NULL;
19     }
20     heap->nodes = (HeapNode *)malloc((size_t)capacity * sizeof(HeapNode));
21     heap->pos = (int *)malloc((size_t)capacity * sizeof(int));
22     if (heap->nodes == NULL || heap->pos == NULL) {
23         free(heap->nodes);
24         free(heap->pos);
25         free(heap);
26         return NULL;
27     }
28     for (i = 0; i < capacity; i++) {
29         heap->pos[i] = -1;
30     }
31     heap->size = 0;
32     heap->capacity = capacity;
33     return heap;
34 }
35
36 int minheap_is_empty(const MinHeap *heap)
37 {
38     return heap->size == 0;
39 }
40
41 int minheap_contains(const MinHeap *heap, int vertex_index)
42 {
43     if (vertex_index < 0 || vertex_index >= heap->capacity) {
44         return 0;
45     }
46     return heap->pos[vertex_index] != -1;
47 }
48
49 /* Swaps two heap slots and keeps the position index consistent. */
50 static void minheap_swap(MinHeap *heap, int a, int b)
51 {
52     HeapNode tmp = heap->nodes[a];
53
54     heap->nodes[a] = heap->nodes[b];
55     heap->nodes[b] = tmp;

```

```

56     heap->pos[heap->nodes[a].vertex_index] = a;
57     heap->pos[heap->nodes[b].vertex_index] = b;
58 }
59
60 /* Moves nodes[i] up while it is smaller than its parent. */
61 static void minheap_sift_up(MinHeap *heap, int i)
62 {
63     while (i > 0) {
64         int parent = (i - 1) / 2;
65         if (heap->nodes[i].dist >= heap->nodes[parent].dist) {
66             break;
67         }
68         minheap_swap(heap, i, parent);
69         i = parent;
70     }
71 }
72
73 void minheap_heapify(MinHeap *heap, int i)
74 {
75     for (;;) {
76         int left = 2 * i + 1;
77         int right = 2 * i + 2;
78         int smallest = i;
79
80         if (left < heap->size
81             && heap->nodes[left].dist < heap->nodes[smallest].dist) {
82             smallest = left;
83         }
84         if (right < heap->size
85             && heap->nodes[right].dist < heap->nodes[smallest].dist) {
86             smallest = right;
87         }
88         if (smallest == i) {
89             break;
90         }
91         minheap_swap(heap, i, smallest);
92         i = smallest;
93     }
94 }
95
96 int minheap_insert(MinHeap *heap, int vertex_index, long dist)
97 {
98     if (vertex_index < 0 || vertex_index >= heap->capacity
99         || heap->size >= heap->capacity
100         || heap->pos[vertex_index] != -1) {
101         return -1;
102     }
103     heap->nodes[heap->size].vertex_index = vertex_index;
104     heap->nodes[heap->size].dist = dist;
105     heap->pos[vertex_index] = heap->size;
106     heap->size++;
107     minheap_sift_up(heap, heap->size - 1);
108     return 0;
109 }
110
111 HeapNode minheap_extract_min(MinHeap *heap)
112 {
113     HeapNode min = heap->nodes[0];
114
115     heap->pos[min.vertex_index] = -1;
116     heap->size--;
117     if (heap->size > 0) {
118         heap->nodes[0] = heap->nodes[heap->size];
119         heap->pos[heap->nodes[0].vertex_index] = 0;
120         minheap_heapify(heap, 0);
121     }
122     return min;
123 }
124
125 int minheap_decrease_key(MinHeap *heap, int vertex_index, long new_dist)

```

```

126 {
127     int i;
128
129     if (vertex_index < 0 || vertex_index >= heap->capacity) {
130         return -1;
131     }
132     i = heap->pos[vertex_index];
133     if (i == -1 || new_dist >= heap->nodes[i].dist) {
134         return -1;
135     }
136     heap->nodes[i].dist = new_dist;
137     minheap_sift_up(heap, i);
138     return 0;
139 }
140
141 void minheap_free(MinHeap *heap)
142 {
143     if (heap == NULL) {
144         return;
145     }
146     free(heap->nodes);
147     free(heap->pos);
148     free(heap);
149 }

```

A.9 src/dijkstra.h

```

1 /*
2  * dijkstra.h
3  * Min-heap-optimized Dijkstra shortest-path search with route
4  * reconstruction. Blocked roads (active == 0) are skipped, a HashSet
5  * records finalized intersections so none is processed twice, and the
6  * search stops early once the destination is finalized.
7  */
8 #ifndef DIJKSTRA_H
9 #define DIJKSTRA_H
10
11 #include "graph.h"
12
13 /* A reconstructed evacuation route. */
14 typedef struct Route {
15     int src_id;
16     int dst_id;
17     int *intersection_ids; /* src ... dst, in travel order */
18     int num_intersections; /* 0 when unreachable */
19     long total_cost; /* -1 when unreachable */
20     int reachable; /* 1 = route exists, 0 = unreachable */
21 } Route;
22
23 /* Work counters, filled when a non-NULL pointer is passed in. */
24 typedef struct DijkstraStats {
25     int vertices_processed; /* extract-min operations performed */
26     long edges_relaxed; /* active edges examined */
27 } DijkstraStats;
28
29 /* Computes the minimum-cost route from src_id to dst_id.
30  * Returns NULL if either ID is not an intersection (or on allocation
31  * failure). An unreachable destination returns a Route with
32  * reachable == 0 – that is a valid answer, not an error.
33  * The caller frees the result with route_free(). */
34 Route *dijkstra_shortest_route(const Graph *g, int src_id, int dst_id,
35                               DijkstraStats *stats);
36
37 /* Number of intersections reachable from src_id over active roads,
38  * including src_id itself. Returns -1 for an unknown source. */
39 long dijkstra_count_reachable(const Graph *g, int src_id);
40
41 /* Deep-copies a route (used by the route cache). */
42 Route *route_clone(const Route *route);
43

```

```

44 /* Prints "id -> id -> ... (total cost C)" or an unreachable notice. */
45 void route_print(const Route *route);
46
47 void route_free(Route *route);
48
49 #endif /* DIJKSTRA_H */

```

A.10 src/dijkstra.c

```

1 /*
2  * dijkstra.c
3  * Dijkstra's algorithm over the HashMap-based adjacency list, driven by
4  * the binary min-heap (insert / extract-min / decrease-key) and guarded
5  * by a HashSet of already-finalized intersections.
6  */
7 #include <stdio.h>
8 #include <stdlib.h>
9 #include <limits.h>
10 #include "dijkstra.h"
11 #include "minheap.h"
12 #include "hashset.h"
13
14 #define DIST_INF LONG_MAX
15
16 /*
17  * Core search shared by the two public functions. Fills dist[] and
18  * prev[] (both indexed by dense vertex index; prev holds the dense index
19  * of the predecessor, or -1). If dst_index >= 0 the search stops as soon
20  * as that vertex is finalized; pass -1 to settle every reachable vertex.
21  * Returns 0 on success, -1 on allocation failure.
22  */
23 static int dijkstra_core(const Graph *g, int src_index, int dst_index,
24                          long *dist, int *prev, DijkstraStats *stats)
25 {
26     int n = g->num_vertices;
27     MinHeap *heap = minheap_create(n);
28     HashSet *processed = hashset_create(n);
29     int i;
30
31     if (heap == NULL || processed == NULL) {
32         minheap_free(heap);
33         hashset_free(processed);
34         return -1;
35     }
36     for (i = 0; i < n; i++) {
37         dist[i] = DIST_INF;
38         prev[i] = -1;
39     }
40     dist[src_index] = 0;
41     minheap_insert(heap, src_index, 0);
42
43     while (!minheap_is_empty(heap)) {
44         HeapNode min = minheap_extract_min(heap);
45         Vertex *u = g->by_index[min.vertex_index];
46         Edge *e;
47
48         /* The HashSet guarantees each intersection is finalized at most
49          * once, so cycles in the road network cannot cause re-work. */
50         if (hashset_contains(processed, min.vertex_index)) {
51             continue;
52         }
53         hashset_add(processed, min.vertex_index);
54         if (stats != NULL) {
55             stats->vertices_processed++;
56         }
57         if (min.vertex_index == dst_index) {
58             break; /* destination finalized: its distance is optimal */
59         }
60         for (e = u->edges; e != NULL; e = e->next) {
61             Vertex *v;

```

```

62         long candidate;
63
64         if (!e->active) {
65             continue; /* blocked road */
66         }
67         v = graph_get_vertex(g, e->dest_id);
68         if (hashset_contains(processed, v->index)) {
69             continue;
70         }
71         if (stats != NULL) {
72             stats->edges_relaxed++;
73         }
74         candidate = dist[min.vertex_index] + e->weight;
75         if (candidate < dist[v->index]) {
76             dist[v->index] = candidate;
77             prev[v->index] = min.vertex_index;
78             if (minheap_contains(heap, v->index)) {
79                 minheap_decrease_key(heap, v->index, candidate);
80             } else {
81                 minheap_insert(heap, v->index, candidate);
82             }
83         }
84     }
85 }
86 minheap_free(heap);
87 hashset_free(processed);
88 return 0;
89 }
90
91 /* Walks prev[] backwards from the destination and reverses the result
92  * into a Route holding external intersection IDs in travel order. */
93 static Route *reconstruct_route(const Graph *g, int src_id, int dst_id,
94                                const long *dist, const int *prev,
95                                int dst_index)
96 {
97     Route *route = (Route *)malloc(sizeof(Route));
98     int hops;
99     int at;
100     int i;
101
102     if (route == NULL) {
103         return NULL;
104     }
105     route->src_id = src_id;
106     route->dst_id = dst_id;
107     if (dist[dst_index] == DIST_INF) {
108         route->intersection_ids = NULL;
109         route->num_intersections = 0;
110         route->total_cost = -1;
111         route->reachable = 0;
112         return route;
113     }
114     hops = 1;
115     for (at = dst_index; prev[at] != -1; at = prev[at]) {
116         hops++;
117     }
118     route->intersection_ids = (int *)malloc((size_t)hops * sizeof(int));
119     if (route->intersection_ids == NULL) {
120         free(route);
121         return NULL;
122     }
123     at = dst_index;
124     for (i = hops - 1; i >= 0; i--) {
125         route->intersection_ids[i] = g->by_index[at]->id;
126         at = prev[at];
127     }
128     route->num_intersections = hops;
129     route->total_cost = dist[dst_index];
130     route->reachable = 1;
131     return route;

```



```

132 }
133
134 Route *dijkstra_shortest_route(const Graph *g, int src_id, int dst_id,
135                               DijkstraStats *stats)
136 {
137     Vertex *src = graph_get_vertex(g, src_id);
138     Vertex *dst = graph_get_vertex(g, dst_id);
139     long *dist;
140     int *prev;
141     Route *route;
142
143     if (src == NULL || dst == NULL) {
144         return NULL; /* invalid source or destination */
145     }
146     if (stats != NULL) {
147         stats->vertices_processed = 0;
148         stats->edges_relaxed = 0;
149     }
150     dist = (long *)malloc((size_t)g->num_vertices * sizeof(long));
151     prev = (int *)malloc((size_t)g->num_vertices * sizeof(int));
152     if (dist == NULL || prev == NULL) {
153         free(dist);
154         free(prev);
155         return NULL;
156     }
157     if (dijkstra_core(g, src->index, dst->index, dist, prev, stats) != 0) {
158         free(dist);
159         free(prev);
160         return NULL;
161     }
162     route = reconstruct_route(g, src_id, dst_id, dist, prev, dst->index);
163     free(dist);
164     free(prev);
165     return route;
166 }
167
168 long dijkstra_count_reachable(const Graph *g, int src_id)
169 {
170     Vertex *src = graph_get_vertex(g, src_id);
171     long *dist;
172     int *prev;
173     long count = 0;
174     int i;
175
176     if (src == NULL) {
177         return -1;
178     }
179     dist = (long *)malloc((size_t)g->num_vertices * sizeof(long));
180     prev = (int *)malloc((size_t)g->num_vertices * sizeof(int));
181     if (dist == NULL || prev == NULL) {
182         free(dist);
183         free(prev);
184         return -1;
185     }
186     if (dijkstra_core(g, src->index, -1, dist, prev, NULL) != 0) {
187         free(dist);
188         free(prev);
189         return -1;
190     }
191     for (i = 0; i < g->num_vertices; i++) {
192         if (dist[i] != DIST_INF) {
193             count++;
194         }
195     }
196     free(dist);
197     free(prev);
198     return count;
199 }
200
201 Route *route_clone(const Route *route)

```

```

202 {
203     Route *copy;
204     int i;
205
206     if (route == NULL) {
207         return NULL;
208     }
209     copy = (Route *)malloc(sizeof(Route));
210     if (copy == NULL) {
211         return NULL;
212     }
213     *copy = *route;
214     copy->intersection_ids = NULL;
215     if (route->num_intersections > 0) {
216         copy->intersection_ids =
217             (int *)malloc((size_t)route->num_intersections * sizeof(int));
218         if (copy->intersection_ids == NULL) {
219             free(copy);
220             return NULL;
221         }
222         for (i = 0; i < route->num_intersections; i++) {
223             copy->intersection_ids[i] = route->intersection_ids[i];
224         }
225     }
226     return copy;
227 }
228
229 void route_print(const Route *route)
230 {
231     int i;
232
233     if (route == NULL) {
234         printf("(no route: invalid query)\n");
235         return;
236     }
237     if (!route->reachable) {
238         printf("Destination %d is UNREACHABLE from %d over active roads.\n",
239             route->dst_id, route->src_id);
240         return;
241     }
242     printf("Route: ");
243     for (i = 0; i < route->num_intersections; i++) {
244         if (i > 0) {
245             printf(" -> ");
246         }
247         printf("%d", route->intersection_ids[i]);
248     }
249     printf(" (total travel cost %ld, %d intersections)\n",
250         route->total_cost, route->num_intersections);
251 }
252
253 void route_free(Route *route)
254 {
255     if (route == NULL) {
256         return;
257     }
258     free(route->intersection_ids);
259     free(route);
260 }

```

A.11 src/route_cache.h

```

1 /*
2  * route_cache.h
3  * Hash-table cache of computed routes for repeated source-to-destination
4  * queries. Every entry remembers the graph version it was computed
5  * against; when a road weight or status changes the graph version
6  * increases, so stale entries are detected on lookup and evicted.
7  */
8 #ifndef ROUTE_CACHE_H

```

```

9 #define ROUTE_CACHE_H
10
11 #include "graph.h"
12 #include "dijkstra.h"
13
14 typedef struct RouteCacheEntry {
15     int src_id;
16     int dst_id;
17     unsigned long graph_version; /* version the route was computed at */
18     Route *route; /* owned deep copy */
19     struct RouteCacheEntry *next;
20 } RouteCacheEntry;
21
22 typedef struct RouteCache {
23     RouteCacheEntry **buckets;
24     int num_buckets; /* power of two */
25     int size;
26     /* statistics, useful for tests and benchmarks */
27     unsigned long hits;
28     unsigned long misses;
29     unsigned long stale_evictions; /* entries dropped as out of date */
30 } RouteCache;
31
32 /* Creates a cache. Returns NULL on allocation failure. */
33 RouteCache *route_cache_create(int initial_buckets);
34
35 /* Returns the cached route for (src, dst) if present AND computed at the
36 * graph's current version; otherwise returns NULL. A stale entry is
37 * evicted, counted in stale_evictions, and reported as a miss. */
38 const Route *route_cache_lookup(RouteCache *cache, const Graph *g,
39                                 int src_id, int dst_id);
40
41 /* Stores a deep copy of the route, stamped with the graph's current
42 * version. Returns 0 on success, -1 on allocation failure. */
43 int route_cache_store(RouteCache *cache, const Graph *g, const Route *route);
44
45 /* Removes every entry but keeps the cache usable. */
46 void route_cache_clear(RouteCache *cache);
47
48 void route_cache_free(RouteCache *cache);
49
50 /*
51 * High-level query used by the demo, tests and benchmark:
52 * answer from the cache when possible, otherwise run Dijkstra and cache
53 * the result (including "unreachable" results, so repeated queries for
54 * unreachable centers are also fast).
55 * Returns a route owned by the cache (do NOT free it), or NULL when
56 * src/dst is not a known intersection. If from_cache is non-NULL it is
57 * set to 1 on a cache hit and 0 on a miss.
58 */
59 const Route *evac_query(Graph *g, RouteCache *cache, int src_id, int dst_id,
60                         int *from_cache);
61
62 #endif /* ROUTE_CACHE_H */

```

A.12 src/route_cache.c

```

1 /*
2  * route_cache.c
3  * Version-stamped route cache with separate chaining, plus the
4  * cache-aware evac_query() entry point.
5  */
6 #include <stdlib.h>
7 #include "route_cache.h"
8
9 #define CACHE_MIN_BUCKETS 16
10
11 /* Mixes the (src, dst) pair into one hash value. */
12 static unsigned long cache_hash(int src_id, int dst_id)
13 {

```

```

14 unsigned long h = (unsigned long)((unsigned int)src_id * 2654435761u);
15
16 h ^= (unsigned long)((unsigned int)dst_id * 40503u) + (h << 6) + (h >> 2);
17 return h;
18 }
19
20 static int round_up_pow2(int n)
21 {
22     int p = CACHE_MIN_BUCKETS;
23     while (p < n && p < (1 << 30)) {
24         p <= 1;
25     }
26     return p;
27 }
28
29 RouteCache *route_cache_create(int initial_buckets)
30 {
31     RouteCache *cache = (RouteCache *)malloc(sizeof(RouteCache));
32
33     if (cache == NULL) {
34         return NULL;
35     }
36     cache->num_buckets = round_up_pow2(initial_buckets);
37     cache->buckets = (RouteCacheEntry **)calloc((size_t)cache->num_buckets,
38                                                sizeof(RouteCacheEntry *));
39     if (cache->buckets == NULL) {
40         free(cache);
41         return NULL;
42     }
43     cache->size = 0;
44     cache->hits = 0;
45     cache->misses = 0;
46     cache->stale_evictions = 0;
47     return cache;
48 }
49
50 /* Finds an entry without touching statistics or version checks. */
51 static RouteCacheEntry *cache_find_raw(const RouteCache *cache,
52                                       int src_id, int dst_id)
53 {
54     unsigned long idx = cache_hash(src_id, dst_id)
55                       & (unsigned long)(cache->num_buckets - 1);
56     RouteCacheEntry *entry;
57
58     for (entry = cache->buckets[idx]; entry != NULL; entry = entry->next) {
59         if (entry->src_id == src_id && entry->dst_id == dst_id) {
60             return entry;
61         }
62     }
63     return NULL;
64 }
65
66 const Route *route_cache_lookup(RouteCache *cache, const Graph *g,
67                                int src_id, int dst_id)
68 {
69     unsigned long idx = cache_hash(src_id, dst_id)
70                       & (unsigned long)(cache->num_buckets - 1);
71     RouteCacheEntry *entry = cache->buckets[idx];
72     RouteCacheEntry *prev = NULL;
73
74     while (entry != NULL) {
75         if (entry->src_id == src_id && entry->dst_id == dst_id) {
76             if (entry->graph_version == g->version) {
77                 cache->hits++;
78                 return entry->route;
79             }
80             /* Road weights or statuses changed since this route was
81              * computed: the entry is invalid, so evict it. */
82             if (prev == NULL) {
83                 cache->buckets[idx] = entry->next;

```

```

84         } else {
85             prev->next = entry->next;
86         }
87         route_free(entry->route);
88         free(entry);
89         cache->size--;
90         cache->stale_evictions++;
91         break;
92     }
93     prev = entry;
94     entry = entry->next;
95 }
96 cache->misses++;
97 return NULL;
98 }
99
100 int route_cache_store(RouteCache *cache, const Graph *g, const Route *route)
101 {
102     unsigned long idx;
103     RouteCacheEntry *entry;
104     Route *copy = route_clone(route);
105
106     if (copy == NULL) {
107         return -1;
108     }
109     entry = (RouteCacheEntry *)malloc(sizeof(RouteCacheEntry));
110     if (entry == NULL) {
111         route_free(copy);
112         return -1;
113     }
114     entry->src_id = route->src_id;
115     entry->dst_id = route->dst_id;
116     entry->graph_version = g->version;
117     entry->route = copy;
118     idx = cache_hash(route->src_id, route->dst_id)
119         & (unsigned long)(cache->num_buckets - 1);
120     entry->next = cache->buckets[idx];
121     cache->buckets[idx] = entry;
122     cache->size++;
123     return 0;
124 }
125
126 void route_cache_clear(RouteCache *cache)
127 {
128     int i;
129
130     for (i = 0; i < cache->num_buckets; i++) {
131         RouteCacheEntry *entry = cache->buckets[i];
132         while (entry != NULL) {
133             RouteCacheEntry *next = entry->next;
134             route_free(entry->route);
135             free(entry);
136             entry = next;
137         }
138         cache->buckets[i] = NULL;
139     }
140     cache->size = 0;
141 }
142
143 void route_cache_free(RouteCache *cache)
144 {
145     if (cache == NULL) {
146         return;
147     }
148     route_cache_clear(cache);
149     free(cache->buckets);
150     free(cache);
151 }
152
153 const Route *evac_query(Graph *g, RouteCache *cache, int src_id, int dst_id,

```

```

154         int *from_cache)
155 {
156     const Route *cached;
157     Route *fresh;
158
159     if (graph_get_vertex(g, src_id) == NULL
160         || graph_get_vertex(g, dst_id) == NULL) {
161         if (from_cache != NULL) {
162             *from_cache = 0;
163         }
164         return NULL;
165     }
166     cached = route_cache_lookup(cache, g, src_id, dst_id);
167     if (cached != NULL) {
168         if (from_cache != NULL) {
169             *from_cache = 1;
170         }
171         return cached;
172     }
173     fresh = dijkstra_shortest_route(g, src_id, dst_id, NULL);
174     if (fresh == NULL) {
175         return NULL;
176     }
177     if (route_cache_store(cache, g, fresh) != 0) {
178         /* Caching failed; the answer is still valid but cannot be kept.
179          * Leak nothing: free the fresh route and report the failure. */
180         route_free(fresh);
181         return NULL;
182     }
183     route_free(fresh);
184     if (from_cache != NULL) {
185         *from_cache = 0;
186     }
187     /* Return the copy the cache now owns (the miss was already counted
188      * by route_cache_lookup above). */
189     return cache_find_raw(cache, src_id, dst_id)->route;
190 }

```

A.13 src/console_ui.h

```

1  /*
2  * console_ui.h
3  * Small console-presentation helpers for the interactive dashboard:
4  * colours, borders, banners, screen clearing, pausing and safe numeric
5  * input. Pure presentation - no routing logic lives here.
6  *
7  * Colours use SetConsoleTextAttribute on Windows and ANSI escapes
8  * elsewhere, and are disabled automatically when stdout is redirected
9  * to a file, so captured output never contains escape garbage.
10 * Only portable ASCII characters are printed.
11 */
12 #ifndef CONSOLE_UI_H
13 #define CONSOLE_UI_H
14
15 /* Visual styles (mapped to colours where available). */
16 typedef enum {
17     UI_DEFAULT = 0,
18     UI_TITLE,      /* cyan    - section titles          */
19     UI_SUCCESS,    /* green   - success messages        */
20     UI_ERROR,      /* red     - errors / unreachable results */
21     UI_WARN        /* yellow  - warnings / progress notices */
22 } UiStyle;
23
24 /* Detects terminal capabilities. Call once at program start. */
25 void ui_init(void);
26
27 /* 1 when stdout is an interactive terminal (colours/clearing on). */
28 int ui_stdout_is_tty(void);
29
30 /* Switches the current output colour; ui_style(UI_DEFAULT) restores. */

```

```

31 void ui_style(UiStyle style);
32
33 /* Prints one whole line in the given style (adds the newline). */
34 void ui_styled_line(UiStyle style, const char *text);
35
36 /* Prints a full-width border line, e.g. ui_rule('='): +====...====+ */
37 void ui_rule(char fill);
38
39 /* Prints "| <text centered>|" across the full width. */
40 void ui_box_center(UiStyle style, const char *text);
41
42 /* Prints " <label> : <value>" with the label padded for alignment. */
43 void ui_kv(const char *label, const char *value);
44
45 /* Project banner (title, course, author, status line). */
46 void ui_banner(void);
47
48 /* Clears the screen when interactive; no-op when redirected. */
49 void ui_clear(void);
50
51 /* "Press Enter to continue" when stdin is interactive; no-op when
52  * input is piped (keeps scripted runs from desynchronising). */
53 void ui_pause(void);
54
55 /*
56  * Safe line-based numeric input. Both print the prompt, read one line
57  * with fgets and parse it strictly.
58  * Return 0 on success,
59  *      -1 when the line is not a valid number (caller may re-prompt),
60  *      -2 on end of input (caller should cancel the operation).
61  */
62 int ui_read_int(const char *prompt, int *out);
63 int ui_read_long(const char *prompt, long *out);
64
65 #endif /* CONSOLE_UI_H */

```

A.14 src/console_ui.c

```

1  /*
2   * console_ui.c
3   * Console presentation helpers. Windows colours go through
4   * SetConsoleTextAttribute; other platforms use ANSI escapes. Both are
5   * disabled when stdout is not an interactive terminal.
6   */
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <string.h>
10 #include <limits.h>
11 #include "console_ui.h"
12
13 #ifdef _WIN32
14 #include <windows.h>
15 #include <io.h>
16 #define UI_ISATTY(fd) _isatty(fd)
17 #else
18 #include <unistd.h>
19 #define UI_ISATTY(fd) isatty(fd)
20 #endif
21
22 #define UI_WIDTH 78 /* total width incl. the border chars */
23 #define UI_INNER (UI_WIDTH - 2)
24
25 static int stdout_tty = 0;
26 static int stdin_tty = 0;
27
28 #ifdef _WIN32
29 static HANDLE console_handle;
30 static WORD default_attrs = 7;
31 #endif
32

```

```

33 void ui_init(void)
34 {
35     stdout_tty = UI_ISATTY(1) ? 1 : 0;
36     stdin_tty = UI_ISATTY(0) ? 1 : 0;
37 #ifdef _WIN32
38     if (stdout_tty) {
39         CONSOLE_SCREEN_BUFFER_INFO info;
40         console_handle = GetStdHandle(STD_OUTPUT_HANDLE);
41         if (GetConsoleScreenBufferInfo(console_handle, &info)) {
42             default_attrs = info.wAttributes;
43         }
44     }
45 #endif
46 }
47
48 int ui_stdout_is_tty(void)
49 {
50     return stdout_tty;
51 }
52
53 void ui_style(UiStyle style)
54 {
55     if (!stdout_tty) {
56         return;
57     }
58 #ifdef _WIN32
59     {
60         WORD attrs = default_attrs;
61         switch (style) {
62             case UI_TITLE:
63                 attrs = FOREGROUND_GREEN | FOREGROUND_BLUE | FOREGROUND_INTENSITY;
64                 break;
65             case UI_SUCCESS:
66                 attrs = FOREGROUND_GREEN | FOREGROUND_INTENSITY;
67                 break;
68             case UI_ERROR:
69                 attrs = FOREGROUND_RED | FOREGROUND_INTENSITY;
70                 break;
71             case UI_WARN:
72                 attrs = FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_INTENSITY;
73                 break;
74             default:
75                 attrs = default_attrs;
76                 break;
77         }
78         SetConsoleTextAttribute(console_handle, attrs);
79     }
80 #else
81     switch (style) {
82         case UI_TITLE: fputs("\033[1;36m", stdout); break;
83         case UI_SUCCESS: fputs("\033[1;32m", stdout); break;
84         case UI_ERROR: fputs("\033[1;31m", stdout); break;
85         case UI_WARN: fputs("\033[1;33m", stdout); break;
86         default: fputs("\033[0m", stdout); break;
87     }
88 #endif
89 }
90
91 void ui_styled_line(UiStyle style, const char *text)
92 {
93     ui_style(style);
94     fputs(text, stdout);
95     ui_style(UI_DEFAULT);
96     fputc('\n', stdout);
97 }
98
99 void ui_rule(char fill)
100 {
101     int i;
102

```



```

103     fputc('+', stdout);
104     for (i = 0; i < UI_INNER; i++) {
105         fputc(fill, stdout);
106     }
107     fputs("+\n", stdout);
108 }
109
110 void ui_box_center(UiStyle style, const char *text)
111 {
112     int len = (int)strlen(text);
113     int left;
114     int right;
115     int i;
116
117     if (len > UI_INNER) {
118         len = UI_INNER; /* truncate over-long lines defensively */
119     }
120     left = (UI_INNER - len) / 2;
121     right = UI_INNER - len - left;
122     fputc('|', stdout);
123     for (i = 0; i < left; i++) {
124         fputc(' ', stdout);
125     }
126     ui_style(style);
127     for (i = 0; i < len; i++) {
128         fputc(text[i], stdout);
129     }
130     ui_style(UI_DEFAULT);
131     for (i = 0; i < right; i++) {
132         fputc(' ', stdout);
133     }
134     fputs("| \n", stdout);
135 }
136
137 void ui_kv(const char *label, const char *value)
138 {
139     printf(" %-22s: %s\n", label, value);
140 }
141
142 void ui_banner(void)
143 {
144     ui_rule('=');
145     ui_box_center(UI_TITLE, "CSA0312 - DATA STRUCTURES");
146     ui_box_center(UI_TITLE, "INTELLIGENT EMERGENCY EVACUATION ROUTING SYSTEM");
147     ui_box_center(UI_DEFAULT, "Athisaya U - 192571001");
148     ui_rule('=');
149     ui_styled_line(UI_SUCCESS,
150         " Status: graph engine ready [HashMap + HashSet + Min-Heap + "
151         " Dijkstra + cache]");
152 }
153
154 void ui_clear(void)
155 {
156     if (!stdout_tty) {
157         return;
158     }
159 #ifdef _WIN32
160     (void)system("cls");
161 #else
162     (void)system("clear");
163 #endif
164 }
165
166 void ui_pause(void)
167 {
168     char buf[128];
169
170     if (!stdin_tty) {
171         return;
172     }

```

```

173     ui_style(UI_WARN);
174     fputs(" Press Enter to continue...", stdout);
175     ui_style(UI_DEFAULT);
176     fflush(stdout);
177     if (fgets(buf, (int)sizeof(buf), stdin) == NULL) {
178         clearerr(stdin);
179     }
180 }
181
182 /* Reads one line and parses a long strictly (whole line must be a
183  * number, surrounding whitespace allowed). */
184 static int read_long_line(const char *prompt, long *out)
185 {
186     char buf[128];
187     char *end;
188     char *p;
189     long value;
190
191     ui_style(UI_TITLE);
192     fputs(prompt, stdout);
193     ui_style(UI_DEFAULT);
194     fputs(": ", stdout);
195     fflush(stdout);
196     if (fgets(buf, (int)sizeof(buf), stdin) == NULL) {
197         return -2;
198     }
199     p = buf;
200     while (*p == ' ' || *p == '\t') {
201         p++;
202     }
203     if (*p == '\0' || *p == '\n') {
204         return -1;
205     }
206     value = strtol(p, &end, 10);
207     while (*end == ' ' || *end == '\t' || *end == '\r' || *end == '\n') {
208         end++;
209     }
210     if (*end != '\0') {
211         return -1; /* trailing junk such as "12abc" */
212     }
213     *out = value;
214     return 0;
215 }
216
217 int ui_read_long(const char *prompt, long *out)
218 {
219     return read_long_line(prompt, out);
220 }
221
222 int ui_read_int(const char *prompt, int *out)
223 {
224     long value;
225     int rc = read_long_line(prompt, &value);
226
227     if (rc != 0) {
228         return rc;
229     }
230     if (value < INT_MIN || value > INT_MAX) {
231         return -1;
232     }
233     *out = (int)value;
234     return 0;
235 }

```

A.15 src/main.c

```

1 /*
2  * main.c
3  * Intelligent Emergency Evacuation Routing System - CSA0312 Slot B.
4  */

```

```

5  * Modes:
6  *   evacsim demo                - narrated small-city walkthrough
7  *   evacsim bench [csv_path]    - large-network benchmark (>8000
8  *                               intersections, >20000 logical roads)
9  *   evacsim fail [txt_path]     - 30% road-failure experiment
10 *
11 * Timing uses standard C clock(); graph generation and file writing are
12 * kept outside every timed region.
13 */
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <string.h>
17 #include <time.h>
18 #include "graph.h"
19 #include "dijkstra.h"
20 #include "route_cache.h"
21 #include "console_ui.h"
22
23 /* ----- */
24 /* Deterministic PRNG (xorshift32). rand() is avoided because its
25 /* RAND_MAX and sequence differ between C libraries; this generator
26 /* reproduces the exact same synthetic city on every platform.
27 /* ----- */
28
29 #define CITY_SEED 20260831UL
30
31 static unsigned long rng_state = CITY_SEED;
32
33 static void rng_seed(unsigned long seed)
34 {
35     rng_state = (seed == 0) ? 1 : seed;
36 }
37
38 static unsigned long rng_next(void)
39 {
40     rng_state ^= (rng_state << 13) & 0xFFFFFFFFUL;
41     rng_state ^= rng_state >> 17;
42     rng_state ^= (rng_state << 5) & 0xFFFFFFFFUL;
43     rng_state &= 0xFFFFFFFFUL;
44     return rng_state;
45 }
46
47 static int rng_range(int n)
48 {
49     return (int)(rng_next() % (unsigned long)n);
50 }
51
52 /* ----- */
53 /* Synthetic city generation (deterministic, fixed seed)
54 /* ----- */
55
56 #define CITY_VERTICES 8200
57 #define CITY_ROADS 21000 /* logical bidirectional roads */
58 #define MAX_WEIGHT 100
59 #define NUM_QUERIES 10
60
61 typedef struct RoadList {
62     int *u;
63     int *v;
64     long count;
65 } RoadList;
66
67 /*
68 * Builds a connected city: vertices 0..CITY_VERTICES-1; first a random
69 * spanning tree (each vertex i>0 links to a random earlier vertex, which
70 * guarantees every intersection is reachable initially), then extra
71 * random roads until CITY_ROADS logical roads exist. Weights are 1..100.
72 * Every generated road is recorded in `roads` for the failure experiment.
73 */
74 static Graph *build_city(RoadList *roads)

```

```

75 {
76     Graph *g = graph_create();
77     long added = 0;
78     int i;
79
80     if (g == NULL) {
81         return NULL;
82     }
83     roads->u = (int *)malloc((size_t)CITY_ROADS * sizeof(int));
84     roads->v = (int *)malloc((size_t)CITY_ROADS * sizeof(int));
85     roads->count = 0;
86     if (roads->u == NULL || roads->v == NULL) {
87         free(roads->u);
88         free(roads->v);
89         graph_free(g);
90         return NULL;
91     }
92     rng_seed(CITY_SEED);
93     for (i = 0; i < CITY_VERTICES; i++) {
94         graph_add_intersection(g, i);
95     }
96     for (i = 1; i < CITY_VERTICES; i++) {
97         int parent = rng_range(i);
98         long w = 1 + rng_range(MAX_WEIGHT);
99         if (graph_add_road(g, i, parent, w) == GRAPH_OK) {
100             roads->u[roads->count] = i;
101             roads->v[roads->count] = parent;
102             roads->count++;
103             added++;
104         }
105     }
106     while (added < CITY_ROADS) {
107         int a = rng_range(CITY_VERTICES);
108         int b = rng_range(CITY_VERTICES);
109         long w = 1 + rng_range(MAX_WEIGHT);
110         if (a != b && graph_add_road(g, a, b, w) == GRAPH_OK) {
111             roads->u[roads->count] = a;
112             roads->v[roads->count] = b;
113             roads->count++;
114             added++;
115         }
116         /* duplicates and self-loops are rejected and simply retried */
117     }
118     return g;
119 }
120
121 /* Fixed, deterministic query set reused by bench and fail modes. */
122 static void make_queries(int src[NUM_QUERIES], int dst[NUM_QUERIES])
123 {
124     int i;
125
126     rng_seed(CITY_SEED ^ 0x5A5A5A5AUL);
127     for (i = 0; i < NUM_QUERIES; i++) {
128         src[i] = rng_range(CITY_VERTICES);
129         dst[i] = rng_range(CITY_VERTICES);
130         if (src[i] == dst[i]) {
131             dst[i] = (dst[i] + 1) % CITY_VERTICES;
132         }
133     }
134 }
135
136 /* ----- */
137 /* Timing helpers (standard C clock(); resolution is about 1 ms, so */
138 /* fast operations are timed in batches and averaged). */
139 /* ----- */
140
141 #define UNCACHED_REPS_PER_PASS 3
142 #define CACHED_LOOKUPS_PER_PASS 20000
143 #define TIMING_PASSES 5
144

```

```

145 /* Average milliseconds of one uncached Dijkstra query (route allocation
146 * and freeing included, since they are part of answering a query). */
147 static double time_uncached_once(Graph *g, int s, int d)
148 {
149     clock_t t0;
150     clock_t t1;
151     int r;
152
153     t0 = clock();
154     for (r = 0; r < UNCACHED_REPS_PER_PASS; r++) {
155         Route *route = dijkstra_shortest_route(g, s, d, NULL);
156         route_free(route);
157     }
158     t1 = clock();
159     return (double)(t1 - t0) * 1000.0 / (double)CLOCKS_PER_SEC
160         / (double)UNCACHED_REPS_PER_PASS;
161 }
162
163 /* Average milliseconds of one cached lookup (cache must be primed). */
164 static double time_cached_once(Graph *g, RouteCache *cache, int s, int d)
165 {
166     clock_t t0;
167     clock_t t1;
168     int r;
169
170     t0 = clock();
171     for (r = 0; r < CACHED_LOOKUPS_PER_PASS; r++) {
172         evac_query(g, cache, s, d, NULL);
173     }
174     t1 = clock();
175     return (double)(t1 - t0) * 1000.0 / (double)CLOCKS_PER_SEC
176         / (double)CACHED_LOOKUPS_PER_PASS;
177 }
178
179 typedef struct TimingSummary {
180     double min_ms;
181     double avg_ms;
182     double max_ms;
183 } TimingSummary;
184
185 static void summarize(const double *samples, int n, TimingSummary *out)
186 {
187     double total = 0.0;
188     int i;
189
190     out->min_ms = samples[0];
191     out->max_ms = samples[0];
192     for (i = 0; i < n; i++) {
193         total += samples[i];
194         if (samples[i] < out->min_ms) {
195             out->min_ms = samples[i];
196         }
197         if (samples[i] > out->max_ms) {
198             out->max_ms = samples[i];
199         }
200     }
201     out->avg_ms = total / (double)n;
202 }
203
204 /* ----- */
205 /* Mode: demo */
206 /* ----- */
207
208 static void demo_query(Graph *g, RouteCache *cache, int s, int d)
209 {
210     int from_cache = 0;
211     const Route *route = evac_query(g, cache, s, d, &from_cache);
212
213     printf("Query %d -> %d [%s]: ", s, d,
214         from_cache ? "cache HIT" : "cache MISS, Dijkstra run");

```

```

215     if (route == NULL) {
216         printf("rejected: invalid intersection ID\n");
217         return;
218     }
219     route_print(route);
220 }
221
222 static int run_demo(void)
223 {
224     Graph *g = graph_create();
225     RouteCache *cache = route_cache_create(16);
226     GraphResult r;
227     int ids[8];
228     int i;
229
230     if (g == NULL || cache == NULL) {
231         graph_free(g);
232         route_cache_free(cache);
233         return 1;
234     }
235     printf("=== Intelligent Emergency Evacuation Routing System: demo ===\n\n");
236
237     printf("[1] Adding 8 intersections (IDs 1..8; 8 stays isolated)...\n");
238     for (i = 0; i < 8; i++) {
239         ids[i] = i + 1;
240         r = graph_add_intersection(g, ids[i]);
241         printf("    add intersection %d: %s\n", ids[i], graph_result_str(r));
242     }
243
244     printf("\n[2] Adding bidirectional roads (weight = safety-adjusted cost)...\n");
245     printf("    road 1-2 w4: %s\n", graph_result_str(graph_add_road(g, 1, 2, 4)));
246     printf("    road 1-3 w1: %s\n", graph_result_str(graph_add_road(g, 1, 3, 1)));
247     printf("    road 3-2 w2: %s\n", graph_result_str(graph_add_road(g, 3, 2, 2)));
248     printf("    road 2-4 w5: %s\n", graph_result_str(graph_add_road(g, 2, 4, 5)));
249     printf("    road 3-4 w12: %s\n", graph_result_str(graph_add_road(g, 3, 4, 12)));
250     printf("    road 4-5 w3: %s\n", graph_result_str(graph_add_road(g, 4, 5, 3)));
251     printf("    road 5-6 w2: %s\n", graph_result_str(graph_add_road(g, 5, 6, 2)));
252     printf("    road 6-7 w1: %s\n", graph_result_str(graph_add_road(g, 6, 7, 1)));
253     printf("    duplicate road 1-2 again: %s\n",
254            graph_result_str(graph_add_road(g, 1, 2, 9)));
255     printf("    negative road 1-4 w-3: %s\n",
256            graph_result_str(graph_add_road(g, 1, 4, -3)));
257
258     printf("\n[3] First query (evacuation center at 4): expect cache MISS.\n");
259     demo_query(g, cache, 1, 4);
260
261     printf("\n[4] Repeated query: expect cache HIT with identical route.\n");
262     demo_query(g, cache, 1, 4);
263
264     printf("\n[5] Disaster update: congestion on road 3-2, weight 2 -> 10.\n");
265     r = graph_update_road_weight(g, 3, 2, 10);
266     printf("    update: %s (graph version is now %lu)\n",
267            graph_result_str(r), g->version);
268     printf("    Same query again: expect cache MISS (entry invalidated).\n");
269     demo_query(g, cache, 1, 4);
270
271     printf("\n[6] Road 2-4 is blocked by debris.\n");
272     r = graph_set_road_status(g, 2, 4, 0);
273     printf("    block: %s\n", graph_result_str(r));
274     demo_query(g, cache, 1, 4);
275
276     printf("\n[7] Road 2-4 is cleared and reopened.\n");
277     r = graph_set_road_status(g, 2, 4, 1);
278     printf("    reopen: %s\n", graph_result_str(r));
279     demo_query(g, cache, 1, 4);
280
281     printf("\n[8] Multiple queries from different disaster sites.\n");
282     demo_query(g, cache, 2, 7);
283     demo_query(g, cache, 3, 6);
284     demo_query(g, cache, 7, 1);

```

```

285
286 printf("\n[9] Unreachable center: intersection 8 has no roads.\n");
287 demo_query(g, cache, 1, 8);
288
289 printf("\n[10] Invalid queries.\n");
290 demo_query(g, cache, 99, 4);
291 demo_query(g, cache, 1, -5);
292
293 printf("\n[11] Cache statistics: hits=%lu misses=%lu stale_evictions=%lu\n",
294         cache->hits, cache->misses, cache->stale_evictions);
295
296 printf("\nDemo finished; freeing all memory.\n");
297 route_cache_free(cache);
298 graph_free(g);
299 return 0;
300 }
301
302 /* ----- */
303 /* Mode: bench */
304 /* ----- */
305
306 static int run_bench(const char *csv_path)
307 {
308     Graph *g;
309     RoadList roads;
310     RouteCache *cache;
311     int src[NUM_QUERIES];
312     int dst[NUM_QUERIES];
313     double uncached[NUM_QUERIES][TIMING_PASSES];
314     double cached[NUM_QUERIES][TIMING_PASSES];
315     long cost[NUM_QUERIES];
316     int hops[NUM_QUERIES];
317     int reach[NUM_QUERIES];
318     FILE *csv;
319     int q;
320     int pass;
321
322     printf("Generating deterministic city (seed %lu): %d intersections, "
323           "%d logical roads...\n", CITY_SEED, CITY_VERTICES, CITY_ROADS);
324     g = build_city(&roads);
325     if (g == NULL) {
326         fprintf(stderr, "error: city generation failed (out of memory)\n");
327         return 1;
328     }
329     cache = route_cache_create(64);
330     if (cache == NULL) {
331         fprintf(stderr, "error: cache creation failed\n");
332         free(roads.u);
333         free(roads.v);
334         graph_free(g);
335         return 1;
336     }
337     printf("Graph ready: %d vertices, %ld logical roads (%ld directed edges).\n",
338           g->num_vertices, g->num_logical_roads, g->num_logical_roads * 2);
339     make_queries(src, dst);
340
341     /* ----- timed region: routing only, no generation, no I/O ---- */
342     for (q = 0; q < NUM_QUERIES; q++) {
343         Route *route;
344
345         for (pass = 0; pass < TIMING_PASSES; pass++) {
346             uncached[q][pass] = time_uncached_once(g, src[q], dst[q]);
347         }
348         route = dijkstra_shortest_route(g, src[q], dst[q], NULL);
349         reach[q] = route->reachable;
350         cost[q] = route->total_cost;
351         hops[q] = route->num_intersections;
352         route_free(route);
353
354         evac_query(g, cache, src[q], dst[q], NULL); /* prime the cache */

```

```

355     for (pass = 0; pass < TIMING_PASSES; pass++) {
356         cached[q][pass] = time_cached_once(g, cache, src[q], dst[q]);
357     }
358 }
359 /* ----- end of timed region ----- */
360
361 csv = fopen(csv_path, "w");
362 if (csv == NULL) {
363     fprintf(stderr, "error: cannot write %s\n", csv_path);
364     route_cache_free(cache);
365     free(roads.u);
366     free(roads.v);
367     graph_free(g);
368     return 1;
369 }
370 fprintf(csv, "record,query,src,dst,reachable,total_cost,intersections,"
371         "passes,min_ms,avg_ms,max_ms\n");
372 fprintf(csv, "meta,vertices,%d,,,,,\n", g->num_vertices);
373 fprintf(csv, "meta,logical_roads,%d,,,,,\n", g->num_logical_roads);
374 fprintf(csv, "meta,seed,%lu,,,,,\n", CITY_SEED);
375 fprintf(csv, "meta,uncached_reps_per_pass,%d,,,,,\n",
376         UNCACHED_REPS_PER_PASS);
377 fprintf(csv, "meta,cached_lookups_per_pass,%d,,,,,\n",
378         CACHED_LOOKUPS_PER_PASS);
379 for (q = 0; q < NUM_QUERIES; q++) {
380     TimingSummary su;
381     TimingSummary sc;
382
383     summarize(uncached[q], TIMING_PASSES, &su);
384     summarize(cached[q], TIMING_PASSES, &sc);
385     fprintf(csv, "uncached,%d,%d,%d,%d,%ld,%d,%d,%d,%d,%d\n",
386             q + 1, src[q], dst[q], reach[q], cost[q], hops[q],
387             TIMING_PASSES, su.min_ms, su.avg_ms, su.max_ms);
388     fprintf(csv, "cached,%d,%d,%d,%d,%ld,%d,%d,%d,%d,%d\n",
389             q + 1, src[q], dst[q], reach[q], cost[q], hops[q],
390             TIMING_PASSES, sc.min_ms, sc.avg_ms, sc.max_ms);
391     printf("Query %2d: %5d -> %5d cost=%6ld hops=%4d "
392            "uncached avg %.3f ms cached avg %.6f ms\n",
393            q + 1, src[q], dst[q], cost[q], hops[q],
394            su.avg_ms, sc.avg_ms);
395 }
396 fclose(csv);
397 printf("Benchmark written to %s\n", csv_path);
398 printf("Cache stats: hits=%lu misses=%lu stale evictions=%lu\n",
399        cache->hits, cache->misses, cache->stale_evictions);
400
401 route_cache_free(cache);
402 free(roads.u);
403 free(roads.v);
404 graph_free(g);
405 return 0;
406 }
407
408 /* ----- */
409 /* Mode: fail (30% of logical roads become unavailable) */
410 /* ----- */
411
412 static int run_fail(const char *txt_path)
413 {
414     Graph *g;
415     RoadList roads;
416     RouteCache *cache;
417     int src[NUM_QUERIES];
418     int dst[NUM_QUERIES];
419     long cost_before[NUM_QUERIES];
420     long cost_after[NUM_QUERIES];
421     int hops_before[NUM_QUERIES];
422     int hops_after[NUM_QUERIES];
423     int reach_before[NUM_QUERIES];
424     int reach_after[NUM_QUERIES];

```



```

425 double t_before[NUM_QUERIES];
426 double t_after[NUM_QUERIES];
427 long *order;
428 long roads_to_block;
429 long reachable_before;
430 long reachable_after;
431 unsigned long stale_before_rerun;
432 int reachable_queries_before = 0;
433 int reachable_queries_after = 0;
434 FILE *out;
435 long i;
436 int q;
437
438 printf("Building the same deterministic city as the benchmark...\n");
439 g = build_city(&roads);
440 if (g == NULL) {
441     fprintf(stderr, "error: city generation failed (out of memory)\n");
442     return 1;
443 }
444 cache = route_cache_create(64);
445 order = (long *)malloc((size_t)roads.count * sizeof(long));
446 if (cache == NULL || order == NULL) {
447     fprintf(stderr, "error: out of memory\n");
448     free(order);
449     route_cache_free(cache);
450     free(roads.u);
451     free(roads.v);
452     graph_free(g);
453     return 1;
454 }
455 make_queries(src, dst);
456
457 /* ---- baseline: timings, routes, connectivity, primed cache ---- */
458 for (q = 0; q < NUM_QUERIES; q++) {
459     Route *route;
460
461     t_before[q] = time_uncached_once(g, src[q], dst[q]);
462     route = dijkstra_shortest_route(g, src[q], dst[q], NULL);
463     reach_before[q] = route->reachable;
464     cost_before[q] = route->total_cost;
465     hops_before[q] = route->num_intersections;
466     if (route->reachable) {
467         reachable_queries_before++;
468     }
469     route_free(route);
470     evac_query(g, cache, src[q], dst[q], NULL); /* fill the cache */
471 }
472 reachable_before = dijkstra_count_reachable(g, src[0]);
473
474 /* ---- deterministically block exactly 30% of logical roads ----- */
475 roads_to_block = (roads.count * 30L) / 100L;
476 for (i = 0; i < roads.count; i++) {
477     order[i] = i;
478 }
479 rng_seed(CITY_SEED ^ 0xC0FFEEUL);
480 for (i = roads.count - 1; i > 0; i--) { /* Fisher-Yates shuffle */
481     long j = (long)(rng_next() % (unsigned long)(i + 1));
482     long tmp = order[i];
483     order[i] = order[j];
484     order[j] = tmp;
485 }
486 for (i = 0; i < roads_to_block; i++) {
487     graph_set_road_status(g, roads.u[order[i]], roads.v[order[i]], 0);
488 }
489 printf("Blocked %ld of %ld logical roads (30%%).\n",
490        roads_to_block, roads.count);
491
492 /* ---- rerun the same queries on the damaged network ----- */
493 stale_before_rerun = cache->stale_evictions;
494 for (q = 0; q < NUM_QUERIES; q++) {

```

```

495     Route *route;
496
497     t_after[q] = time_uncached_once(g, src[q], dst[q]);
498     route = dijkstra_shortest_route(g, src[q], dst[q], NULL);
499     reach_after[q] = route->reachable;
500     cost_after[q] = route->total_cost;
501     hops_after[q] = route->num_intersections;
502     if (route->reachable) {
503         reachable_queries_after++;
504     }
505     route_free(route);
506     evac_query(g, cache, src[q], dst[q], NULL); /* stale -> evict */
507 }
508 reachable_after = dijkstra_count_reachable(g, src[0]);
509
510 /* ---- write the analysis (outside all timed regions) ----- */
511 out = fopen(txt_path, "w");
512 if (out == NULL) {
513     fprintf(stderr, "error: cannot write %s\n", txt_path);
514     free(order);
515     route_cache_free(cache);
516     free(roads.u);
517     free(roads.v);
518     graph_free(g);
519     return 1;
520 }
521 fprintf(out, "30%% ROAD FAILURE EXPERIMENT\n");
522 fprintf(out, "=====\n\n");
523 fprintf(out, "Network: %d intersections, %ld logical bidirectional roads\n",
524         g->num_vertices, g->num_logical_roads);
525 fprintf(out, "Deterministic generation seed: %lu\n", CITY_SEED);
526 fprintf(out, "Roads blocked: %ld of %ld (exactly 30%%), selected by a\n",
527         roads_to_block, roads.count);
528 fprintf(out, "Fisher-Yates shuffle seeded with %lu ^ 0xC0FFEE.\n\n",
529         CITY_SEED);
530 fprintf(out, "Timing: standard C clock(), avg of %d Dijkstra runs per "
531         "query,\nresolution about 1 ms; graph generation and file "
532         "writing excluded.\n\n", UNCACHED_REPS_PER_PASS);
533
534 fprintf(out, "PER-QUERY RESULTS (before -> after blocking)\n");
535 fprintf(out, "%-3s %-6s %-6s | %-11s %-11s | %-9s %-9s | %-6s %-6s | %-9s %-9s\n",
536         "Q", "src", "dst", "reach_pre", "reach_post", "cost_pre",
537         "cost_post", "hops0", "hops1", "ms_pre", "ms_post");
538 for (q = 0; q < NUM_QUERIES; q++) {
539     fprintf(out, "%-3d %-6d %-6d | %-11s %-11s | %-9ld %-9ld | %-6d %-6d | %-9.3f %-9.3f\n",
540 n",
541         q + 1, src[q], dst[q],
542         reach_before[q] ? "yes" : "NO",
543         reach_after[q] ? "yes" : "NO",
544         cost_before[q], cost_after[q],
545         hops_before[q], hops_after[q],
546         t_before[q], t_after[q]);
547 }
548
549 fprintf(out, "\nSUMMARY\n");
550 fprintf(out, "Reachable destination queries: %d/%d before, %d/%d after.\n",
551         reachable_queries_before, NUM_QUERIES,
552         reachable_queries_after, NUM_QUERIES);
553 fprintf(out, "Intersections reachable from intersection %d: %ld of %d "
554         "before,\n%ld of %d after blocking.\n",
555         src[0], reachable_before, g->num_vertices,
556         reachable_after, g->num_vertices);
557 for (q = 0; q < NUM_QUERIES; q++) {
558     if (reach_before[q] && reach_after[q]) {
559         fprintf(out, "Query %d: cost %ld -> %ld (%+ld), route length "
560                 "%d -> %d intersections.\n",
561                 q + 1, cost_before[q], cost_after[q],
562                 cost_after[q] - cost_before[q],
563                 hops_before[q], hops_after[q]);
564     } else if (reach_before[q] && !reach_after[q]) {

```

```

564         fprintf(out, "Query %d: destination became UNREACHABLE after "
565                 "the failures.\n", q + 1);
566     }
567 }
568 fprintf(out, "\nCACHE BEHAVIOUR\n");
569 fprintf(out, "Every route cached before the failures was invalidated: "
570         "blocking\nroads bumped the graph version, so the rerun "
571         "evicted %lu stale\nentries and recomputed (hits=%lu, "
572         "misses=%lu, stale_evictions=%lu).\n",
573         cache->stale_evictions - stale_before_rerun,
574         cache->hits, cache->misses, cache->stale_evictions);
575
576 fclose(out);
577 printf("Analysis written to %s\n", txt_path);
578
579 free(order);
580 route_cache_free(cache);
581 free(roads.u);
582 free(roads.v);
583 graph_free(g);
584 return 0;
585 }
586
587 /* ----- */
588 /* Interactive dashboard (no-argument mode) */
589 /* ----- */
590 /* A presentation layer over the existing engine: it builds a small */
591 /* demonstration city through the normal graph API and lets an */
592 /* evaluator run queries, updates, blockings and the automated modes */
593 /* from a menu. All routing behaviour is the engine's own. */
594 /* ----- */
595
596 /* Same city as the guided demo: intersections 1..8 (8 isolated),
597 * roads 1-2 w4, 1-3 w1, 3-2 w2, 2-4 w5, 3-4 w12, 4-5 w3, 5-6 w2,
598 * 6-7 w1. */
599 static Graph *build_planner_city(void)
600 {
601     Graph *g = graph_create();
602     int i;
603
604     if (g == NULL) {
605         return NULL;
606     }
607     for (i = 1; i <= 8; i++) {
608         graph_add_intersection(g, i);
609     }
610     graph_add_road(g, 1, 2, 4);
611     graph_add_road(g, 1, 3, 1);
612     graph_add_road(g, 3, 2, 2);
613     graph_add_road(g, 2, 4, 5);
614     graph_add_road(g, 3, 4, 12);
615     graph_add_road(g, 4, 5, 3);
616     graph_add_road(g, 5, 6, 2);
617     graph_add_road(g, 6, 7, 1);
618     return g;
619 }
620
621 /* Re-prompts while the input is not a valid number.
622 * Returns 0 on success, -2 when input ended (cancel). */
623 static int read_int_retry(const char *prompt, int *out)
624 {
625     for (;;) {
626         int rc = ui_read_int(prompt, out);
627         if (rc == 0) {
628             return 0;
629         }
630         if (rc == -2) {
631             return -2;
632         }
633         ui_styled_line(UI_ERROR,

```

```

634         " That is not a valid whole number - try again.");
635     }
636 }
637
638 static int read_long_retry(const char *prompt, long *out)
639 {
640     for (;;) {
641         int rc = ui_read_long(prompt, out);
642         if (rc == 0) {
643             return 0;
644         }
645         if (rc == -2) {
646             return -2;
647         }
648         ui_styled_line(UI_ERROR,
649             " That is not a valid whole number - try again.");
650     }
651 }
652
653 static void planner_show_city(const Graph *g)
654 {
655     int i;
656     char buf[128];
657
658     ui_rule('-');
659     ui_box_center(UI_TITLE, "DEMONSTRATION CITY MAP");
660     ui_rule('-');
661     sprintf(buf, "%d", g->num_vertices);
662     ui_kv("Intersections", buf);
663     sprintf(buf, "%ld (bidirectional)", g->num_logical_roads);
664     ui_kv("Logical roads", buf);
665     sprintf(buf, "%lu", g->version);
666     ui_kv("Graph version", buf);
667     printf("\n %-14s %-10s %s\n", "ROAD", "WEIGHT", "STATUS");
668     for (i = 0; i < g->num_vertices; i++) {
669         const Vertex *u = g->by_index[i];
670         const Edge *e;
671         for (e = u->edges; e != NULL; e = e->next) {
672             if (u->id < e->dest_id) { /* print each logical road once */
673                 printf(" %4d - %6d %10ld ", u->id, e->dest_id,
674                     e->weight);
675                 if (e->active) {
676                     ui_styled_line(UI_SUCCESS, "OPEN");
677                 } else {
678                     ui_styled_line(UI_ERROR, "BLOCKED");
679                 }
680             }
681         }
682     }
683     for (i = 0; i < g->num_vertices; i++) {
684         if (g->by_index[i]->degree == 0) {
685             sprintf(buf, " Intersection %d has no roads (isolated).",
686                 g->by_index[i]->id);
687             ui_styled_line(UI_WARN, buf);
688         }
689     }
690 }
691
692 static void planner_find_route(Graph *g, RouteCache *cache)
693 {
694     int src;
695     int dst;
696     int from_cache = 0;
697     const Route *route;
698     char buf[192];
699     char path[512];
700     int i;
701
702     if (read_int_retry(" Source intersection ID", &src) != 0
703         || read_int_retry(" Destination intersection ID", &dst) != 0) {

```

```

704     return;
705 }
706 if (graph_get_vertex(g, src) == NULL) {
707     sprintf(buf, " Intersection %d does not exist in this city.", src);
708     ui_styled_line(UI_ERROR, buf);
709     return;
710 }
711 if (graph_get_vertex(g, dst) == NULL) {
712     sprintf(buf, " Intersection %d does not exist in this city.", dst);
713     ui_styled_line(UI_ERROR, buf);
714     return;
715 }
716 route = evac_query(g, cache, src, dst, &from_cache);
717 if (route == NULL) {
718     ui_styled_line(UI_ERROR, " Query failed unexpectedly.");
719     return;
720 }
721 printf("\n");
722 ui_rule('-');
723 if (!route->reachable) {
724     ui_box_center(UI_ERROR, "DESTINATION UNREACHABLE");
725     ui_rule('-');
726     sprintf(buf, "%d", src);
727     ui_kv("Source", buf);
728     sprintf(buf, "%d", dst);
729     ui_kv("Destination", buf);
730     sprintf(buf,
731         " No open road chain connects %d to %d right now.",
732         src, dst);
733     ui_styled_line(UI_ERROR, buf);
734     ui_kv("Answered from", from_cache ? "route cache (HIT)"
735         : "Dijkstra run (cache MISS)");
736     return;
737 }
738 ui_box_center(UI_SUCCESS, "EVACUATION ROUTE FOUND");
739 ui_rule('-');
740 sprintf(buf, "%d", src);
741 ui_kv("Source", buf);
742 sprintf(buf, "%d", dst);
743 ui_kv("Destination", buf);
744 path[0] = '\0';
745 for (i = 0; i < route->num_intersections; i++) {
746     char piece[32];
747     sprintf(piece, i == 0 ? "%d" : " -> %d",
748         route->intersection_ids[i]);
749     if (strlen(path) + strlen(piece) + 1 < sizeof(path)) {
750         strcat(path, piece);
751     }
752 }
753 ui_kv("Route", path);
754 sprintf(buf, "%ld", route->total_cost);
755 ui_kv("Total travel cost", buf);
756 sprintf(buf, "%d", route->num_intersections);
757 ui_kv("Intersections", buf);
758 ui_kv("Answered from", from_cache ? "route cache (HIT)"
759     : "Dijkstra run (cache MISS)");
760 }
761
762 /* action: 0 = update weight, 1 = block, 2 = reopen. */
763 static void planner_edit_road(Graph *g, int action)
764 {
765     int u;
766     int v;
767     long w = 0;
768     GraphResult rc;
769     char buf[160];
770
771     if (read_int_retry(" First intersection ID", &u) != 0
772         || read_int_retry(" Second intersection ID", &v) != 0) {
773         return;

```

```

774     }
775     if (action == 0) {
776         if (read_long_retry(" New travel cost (>= 0)", &w) != 0) {
777             return;
778         }
779         rc = graph_update_road_weight(g, u, v, w);
780     } else {
781         rc = graph_set_road_status(g, u, v, action == 2 ? 1 : 0);
782     }
783     if (rc != GRAPH_OK) {
784         sprintf(buf, " Operation rejected: %s.", graph_result_str(rc));
785         ui_styled_line(UI_ERROR, buf);
786         return;
787     }
788     if (action == 0) {
789         sprintf(buf, " Road %d-%d now has travel cost %ld.", u, v, w);
790     } else if (action == 1) {
791         sprintf(buf, " Road %d-%d is now BLOCKED in both directions.",
792             u, v);
793     } else {
794         sprintf(buf, " Road %d-%d is OPEN again with its stored cost.",
795             u, v);
796     }
797     ui_styled_line(UI_SUCCESS, buf);
798     sprintf(buf, " Cached routes are now stale (graph version %lu).",
799         g->version);
800     ui_styled_line(UI_WARN, buf);
801 }
802
803 static void planner_cache_stats(const RouteCache *cache, const Graph *g)
804 {
805     char buf[64];
806
807     ui_rule('-');
808     ui_box_center(UI_TITLE, "ROUTE CACHE STATISTICS");
809     ui_rule('-');
810     sprintf(buf, "%lu", cache->hits);
811     ui_kv("Cache hits", buf);
812     sprintf(buf, "%lu", cache->misses);
813     ui_kv("Cache misses", buf);
814     sprintf(buf, "%lu", cache->stale_evictions);
815     ui_kv("Stale evictions", buf);
816     sprintf(buf, "%d", cache->size);
817     ui_kv("Routes stored", buf);
818     sprintf(buf, "%lu", g->version);
819     ui_kv("Graph version", buf);
820 }
821
822 static int run_planner(void)
823 {
824     Graph *g = build_planner_city();
825     RouteCache *cache = route_cache_create(16);
826     int running = 1;
827
828     if (g == NULL || cache == NULL) {
829         graph_free(g);
830         route_cache_free(cache);
831         ui_styled_line(UI_ERROR, " Out of memory.");
832         return 1;
833     }
834     while (running) {
835         int choice;
836         int rc;
837
838         ui_clear();
839         ui_rule('=');
840         ui_box_center(UI_TITLE, "INTERACTIVE EVACUATION ROUTE PLANNER");
841         ui_rule('=');
842         printf(" [1] View intersections and roads\n");
843         printf(" [2] Find an evacuation route\n");

```

```

844     printf("  [3] Update a road weight\n");
845     printf("  [4] Block a road\n");
846     printf("  [5] Reopen a road\n");
847     printf("  [6] View route-cache statistics\n");
848     printf("  [7] Reset the demonstration city\n");
849     printf("  [0] Back to main menu\n");
850     ui_rule('-');
851     rc = ui_read_int("  Choose an option", &choice);
852     if (rc == -2) {
853         break;
854     }
855     if (rc != 0) {
856         ui_styled_line(UI_ERROR,
857             "    Please enter one of the menu numbers.");
858         ui_pause();
859         continue;
860     }
861     printf("\n");
862     switch (choice) {
863     case 1:
864         planner_show_city(g);
865         break;
866     case 2:
867         planner_find_route(g, cache);
868         break;
869     case 3:
870         planner_edit_road(g, 0);
871         break;
872     case 4:
873         planner_edit_road(g, 1);
874         break;
875     case 5:
876         planner_edit_road(g, 2);
877         break;
878     case 6:
879         planner_cache_stats(cache, g);
880         break;
881     case 7:
882         graph_free(g);
883         route_cache_free(cache);
884         g = build_planner_city();
885         cache = route_cache_create(16);
886         if (g == NULL || cache == NULL) {
887             ui_styled_line(UI_ERROR, "    Out of memory.");
888             running = 0;
889             break;
890         }
891         ui_styled_line(UI_SUCCESS,
892             "    Demonstration city rebuilt with its original roads.");
893         break;
894     case 0:
895         running = 0;
896         break;
897     default:
898         ui_styled_line(UI_ERROR,
899             "    Please enter one of the menu numbers.");
900         break;
901     }
902     if (running) {
903         printf("\n");
904         ui_pause();
905     }
906 }
907 graph_free(g);
908 route_cache_free(cache);
909 return 0;
910 }
911
912 static void show_architecture(void)
913 {

```

```

914 ui_rule('-');
915 ui_box_center(UI_TITLE, "SYSTEM ARCHITECTURE AND FEATURES");
916 ui_rule('-');
917 printf(" %-16s %s\n", "MODULE", "ROLE");
918 printf(" %-16s %s\n", "graph.c",
919       "HashMap-based adjacency list, dynamic updates, versioning");
920 printf(" %-16s %s\n", "hashmap.c",
921       "intersection ID -> vertex lookup, O(1) average");
922 printf(" %-16s %s\n", "hashset.c",
923       "processed-intersection set inside Dijkstra");
924 printf(" %-16s %s\n", "minheap.c",
925       "insert / extract-min / heapify / decrease-key, O(log V)");
926 printf(" %-16s %s\n", "dijkstra.c",
927       "O((V+E) log V) shortest routes + route reconstruction");
928 printf(" %-16s %s\n", "route_cache.c",
929       "version-stamped cache for repeated queries");
930 printf(" %-16s %s\n", "console_ui.c",
931       "this dashboard (presentation only)");
932 printf(" %-16s %s\n", "main.c",
933       "program flow: interactive / demo / bench / fail");
934 ui_rule('-');
935 printf(" Key properties:\n");
936 printf(" - roads are bidirectional with non-negative,"
937       " safety-adjusted costs\n");
938 printf(" - blocked roads are skipped, reopening restores the"
939       " stored cost\n");
940 printf(" - every mutation bumps the graph version and invalidates"
941       " the cache\n");
942 printf(" - unreachable evacuation centers are reported, never"
943       " looped on\n");
944 printf(" - benchmark city: 8,200 intersections / 21,000 logical"
945       " roads (seed %lu)\n", (unsigned long)CITY_SEED);
946 ui_styled_line(UI_WARN,
947       " Console-based C data-structures prototype on synthetic data"
948       " - not a\n deployed navigation application.");
949 }
950
951 static int run_interactive(void)
952 {
953     int running = 1;
954
955     while (running) {
956         int choice;
957         int rc;
958
959         ui_clear();
960         ui_banner();
961         printf("\n");
962         printf(" [1] Interactive Evacuation Route Planner\n");
963         printf(" [2] Run Guided System Demonstration\n");
964         printf(" [3] Run Full-Scale Performance Benchmark\n");
965         printf(" [4] Simulate 30%% Road Failure\n");
966         printf(" [5] View System Architecture and Features\n");
967         printf(" [0] Exit\n");
968         ui_rule('-');
969         rc = ui_read_int(" Choose an option", &choice);
970         if (rc == -2) {
971             break;
972         }
973         if (rc != 0) {
974             ui_styled_line(UI_ERROR,
975                 " Please enter one of the menu numbers.");
976             ui_pause();
977             continue;
978         }
979         printf("\n");
980         switch (choice) {
981             case 1:
982                 run_planner();
983                 break;

```



```

984     case 2:
985         ui_clear();
986         run_demo();
987         printf("\n");
988         ui_styled_line(UI_SUCCESS, " Guided demonstration finished.");
989         ui_pause();
990         break;
991     case 3:
992         ui_styled_line(UI_WARN,
993             " Building the 8,200-intersection / 21,000-road city"
994             " and timing");
995         ui_styled_line(UI_WARN,
996             " 10 queries (uncached and cached). This takes a few"
997             " seconds...");
998         printf("\n");
999         if (run_bench("results/benchmark_results.csv") == 0) {
1000             ui_styled_line(UI_SUCCESS,
1001                 " Benchmark finished - see"
1002                 " results/benchmark_results.csv.");
1003         } else {
1004             ui_styled_line(UI_ERROR, " Benchmark failed.");
1005         }
1006         ui_pause();
1007         break;
1008     case 4:
1009         ui_styled_line(UI_WARN,
1010             " Simulating a disaster that blocks 30% of all"
1011             " logical roads...");
1012         printf("\n");
1013         if (run_fail("results/road_failure_analysis.txt") == 0) {
1014             ui_styled_line(UI_SUCCESS,
1015                 " Simulation finished - see"
1016                 " results/road_failure_analysis.txt.");
1017         } else {
1018             ui_styled_line(UI_ERROR, " Simulation failed.");
1019         }
1020         ui_pause();
1021         break;
1022     case 5:
1023         show_architecture();
1024         ui_pause();
1025         break;
1026     case 0:
1027         running = 0;
1028         break;
1029     default:
1030         ui_styled_line(UI_ERROR,
1031             " Please enter one of the menu numbers.");
1032         ui_pause();
1033         break;
1034 }
1035 }
1036 ui_styled_line(UI_SUCCESS,
1037     " All memory released. Stay safe - goodbye!");
1038 return 0;
1039 }
1040
1041 /* ----- */
1042
1043 int main(int argc, char **argv)
1044 {
1045     ui_init();
1046     if (argc < 2) {
1047         return run_interactive();
1048     }
1049     if (strcmp(argv[1], "demo") == 0) {
1050         return run_demo();
1051     }
1052     if (strcmp(argv[1], "bench") == 0) {
1053         return run_bench(argc >= 3 ? argv[2]

```

```

1054         : "results/benchmark_results.csv");
1055     }
1056     if (strcmp(argv[1], "fail") == 0) {
1057         return run_fail(argc >= 3 ? argv[2]
1058             : "results/road_failure_analysis.txt");
1059     }
1060     fprintf(stderr,
1061         "usage: %s [demo | bench [csv_path] | fail [txt_path]]\n"
1062         "        (no argument starts the interactive dashboard)\n",
1063         argv[0]);
1064     return 2;
1065 }

```

A.16 tests/test_runner.c

```

1  /*
2  * test_runner.c
3  * Functional tests for the Intelligent Emergency Evacuation Routing
4  * System. Every test prints expected result, actual result and
5  * PASS/FAIL. Usage:
6  *   test_runner [output_path]
7  * With no argument the report goes to stdout.
8  *
9  * Known test graph (bidirectional roads, weight = travel cost):
10 *
11 *   1 --4-- 2 --5-- 4 --3-- 5
12 *   |      /      /
13 *   1      2      12
14 *   |      /      /
15 *   3 -----
16 *   (6 is an isolated intersection)
17 *
18 * Roads: 1-2 w4, 1-3 w1, 3-2 w2, 2-4 w5, 3-4 w12, 4-5 w3.
19 * Hand-checked shortest route 1 -> 4:
20 *   1-3-2-4 = 1+2+5 = 8   (vs 1-2-4 = 9, 1-3-4 = 13) -> cost 8.
21 */
22 #include <stdio.h>
23 #include <stdlib.h>
24 #include <string.h>
25 #include "../src/graph.h"
26 #include "../src/dijkstra.h"
27 #include "../src/route_cache.h"
28
29 static FILE *out;
30 static int tests_run = 0;
31 static int tests_passed = 0;
32
33 static void record(const char *name, const char *expected,
34                  const char *actual, int pass)
35 {
36     tests_run++;
37     if (pass) {
38         tests_passed++;
39     }
40     fprintf(out, "Test %02d: %-42s\n", tests_run, name);
41     fprintf(out, "    expected: %s\n", expected);
42     fprintf(out, "    actual:   %s\n", actual);
43     fprintf(out, "    result:   %s\n\n", pass ? "PASS" : "FAIL");
44 }
45
46 /* Formats a route as "cost C via a -> b -> c" into buf. */
47 static void format_route(const Route *route, char *buf)
48 {
49     int i;
50     char piece[32];
51
52     if (route == NULL) {
53         strcpy(buf, "NULL (invalid query)");
54         return;
55     }

```

```

56     if (!route->reachable) {
57         strcpy(buf, "unreachable");
58         return;
59     }
60     sprintf(buf, "cost %ld via ", route->total_cost);
61     for (i = 0; i < route->num_intersections; i++) {
62         sprintf(piece, i == 0 ? "%d" : " -> %d", route->intersection_ids[i]);
63         strcat(buf, piece);
64     }
65 }
66
67 /* Compares a route against an expected cost and vertex sequence. */
68 static int route_matches(const Route *route, long cost,
69                          const int *ids, int n)
70 {
71     int i;
72
73     if (route == NULL || !route->reachable || route->total_cost != cost
74         || route->num_intersections != n) {
75         return 0;
76     }
77     for (i = 0; i < n; i++) {
78         if (route->intersection_ids[i] != ids[i]) {
79             return 0;
80         }
81     }
82     return 1;
83 }
84
85 static Graph *build_known_graph(void)
86 {
87     Graph *g = graph_create();
88     int i;
89
90     for (i = 1; i <= 6; i++) {
91         graph_add_intersection(g, i);
92     }
93     graph_add_road(g, 1, 2, 4);
94     graph_add_road(g, 1, 3, 1);
95     graph_add_road(g, 3, 2, 2);
96     graph_add_road(g, 2, 4, 5);
97     graph_add_road(g, 3, 4, 12);
98     graph_add_road(g, 4, 5, 3);
99     /* intersection 6 is left isolated on purpose */
100    return g;
101 }
102
103 int main(int argc, char **argv)
104 {
105     Graph *g;
106     RouteCache *cache;
107     Route *route;
108     const Route *cached;
109     char actual[512];
110     char expected[512];
111     int from_cache;
112     GraphResult gr;
113
114     out = stdout;
115     if (argc >= 2) {
116         out = fopen(argv[1], "w");
117         if (out == NULL) {
118             fprintf(stderr, "error: cannot write %s\n", argv[1]);
119             return 2;
120         }
121     }
122     fprintf(out, "CSA0312 - Intelligent Emergency Evacuation Routing System\n");
123     fprintf(out, "Functional test results\n");
124     fprintf(out, "=====\n");
125     fprintf(out, "Known graph: intersections 1..6 (6 isolated); roads\n");

```

```

126     fprintf(out, "1-2 w4, 1-3 w1, 3-2 w2, 2-4 w5, 3-4 w12, 4-5 w3.\n\n");
127
128     g = build_known_graph();
129     cache = route_cache_create(16);
130
131     /* 1. Known graph with a manually verifiable shortest route ----- */
132     route = dijkstra_shortest_route(g, 1, 4, NULL);
133     format_route(route, actual);
134     record("known shortest route 1 -> 4",
135           "cost 8 via 1 -> 3 -> 2 -> 4", actual,
136           route_matches(route, 8, (const int[]){1, 3, 2, 4}, 4));
137     route_free(route);
138
139     /* 2. Route reconstruction ----- */
140     route = dijkstra_shortest_route(g, 1, 5, NULL);
141     format_route(route, actual);
142     record("route reconstruction 1 -> 5",
143           "cost 11 via 1 -> 3 -> 2 -> 4 -> 5", actual,
144           route_matches(route, 11, (const int[]){1, 3, 2, 4, 5}, 5));
145     route_free(route);
146
147     /* 3. Weight update changes the best route ----- */
148     gr = graph_update_road_weight(g, 3, 2, 10);
149     route = dijkstra_shortest_route(g, 1, 4, NULL);
150     format_route(route, actual);
151     sprintf(expected, "update OK; new best cost 9 via 1 -> 2 -> 4");
152     record("weight update 3-2: 2 -> 10 reroutes 1 -> 4", expected, actual,
153           gr == GRAPH_OK
154           && route_matches(route, 9, (const int[]){1, 2, 4}, 3));
155     route_free(route);
156
157     /* 4. Blocking a road forces an alternative route ----- */
158     gr = graph_set_road_status(g, 2, 4, 0);
159     route = dijkstra_shortest_route(g, 1, 4, NULL);
160     format_route(route, actual);
161     record("blocking road 2-4 reroutes 1 -> 4",
162           "block OK; new best cost 13 via 1 -> 3 -> 4", actual,
163           gr == GRAPH_OK
164           && route_matches(route, 13, (const int[]){1, 3, 4}, 3));
165     route_free(route);
166
167     /* 5. Reopening the road restores the previous route ----- */
168     gr = graph_set_road_status(g, 2, 4, 1);
169     route = dijkstra_shortest_route(g, 1, 4, NULL);
170     format_route(route, actual);
171     record("reopening road 2-4 restores the route",
172           "reopen OK; best cost 9 via 1 -> 2 -> 4 again", actual,
173           gr == GRAPH_OK
174           && route_matches(route, 9, (const int[]){1, 2, 4}, 3));
175     route_free(route);
176
177     /* 6. Invalid source ----- */
178     route = dijkstra_shortest_route(g, 99, 4, NULL);
179     record("invalid source intersection 99",
180           "query rejected (NULL route)",
181           route == NULL ? "NULL (invalid query)" : "a route was returned",
182           route == NULL);
183     route_free(route);
184
185     /* 7. Invalid destination ----- */
186     route = dijkstra_shortest_route(g, 1, 77, NULL);
187     record("invalid destination intersection 77",
188           "query rejected (NULL route)",
189           route == NULL ? "NULL (invalid query)" : "a route was returned",
190           route == NULL);
191     route_free(route);
192
193     /* 8. Duplicate road ----- */
194     gr = graph_add_road(g, 1, 2, 7);
195     record("duplicate road 1-2 rejected",

```

```

196         "duplicate road", graph_result_str(gr),
197         gr == GRAPH_ERR_DUPLICATE_ROAD);
198
199     /* 9. Negative weight rejection ----- */
200     gr = graph_add_road(g, 5, 6, -5);
201     sprintf(actual, "add: %s", graph_result_str(gr));
202     {
203         GraphResult gr2 = graph_update_road_weight(g, 1, 2, -1);
204         char buf[64];
205         sprintf(buf, "; update: %s", graph_result_str(gr2));
206         strcat(actual, buf);
207         record("negative weights rejected (add and update)",
208             "add: negative weight rejected; update: negative weight rejected",
209             actual,
210             gr == GRAPH_ERR_NEGATIVE_WEIGHT
211             && gr2 == GRAPH_ERR_NEGATIVE_WEIGHT);
212     }
213
214     /* 10. Unreachable destination ----- */
215     route = dijkstra_shortest_route(g, 1, 6, NULL);
216     format_route(route, actual);
217     record("unreachable destination 6 (isolated)",
218         "unreachable", actual,
219         route != NULL && !route->reachable);
220     route_free(route);
221
222     /* 11. Multiple source-to-destination queries ----- */
223     {
224         Route *r1 = dijkstra_shortest_route(g, 1, 4, NULL);
225         Route *r2 = dijkstra_shortest_route(g, 1, 5, NULL);
226         Route *r3 = dijkstra_shortest_route(g, 2, 5, NULL);
227         int ok = route_matches(r1, 9, (const int[]){1, 2, 4}, 3)
228             && r2 != NULL && r2->reachable && r2->total_cost == 12
229             && r3 != NULL && r3->reachable && r3->total_cost == 8;
230         sprintf(actual, "1->4 cost %ld, 1->5 cost %ld, 2->5 cost %ld",
231             r1 == NULL ? -99 : r1->total_cost,
232             r2 == NULL ? -99 : r2->total_cost,
233             r3 == NULL ? -99 : r3->total_cost);
234         record("multiple queries 1->4, 1->5, 2->5",
235             "costs 9, 12 and 8", actual, ok);
236         route_free(r1);
237         route_free(r2);
238         route_free(r3);
239     }
240
241     /* 12. Cache miss on first query ----- */
242     cached = evac_query(g, cache, 1, 4, &from_cache);
243     sprintf(actual, "from_cache=%d, %s", from_cache,
244         cached != NULL && cached->reachable ? "route returned"
245             : "no route");
246     record("first cached query 1 -> 4 is a MISS",
247         "from_cache=0, route returned", actual,
248         cached != NULL && from_cache == 0 && cached->reachable
249         && cached->total_cost == 9);
250
251     /* 13. Cache hit on repeated query ----- */
252     cached = evac_query(g, cache, 1, 4, &from_cache);
253     sprintf(actual, "from_cache=%d, cost %ld", from_cache,
254         cached == NULL ? -99 : cached->total_cost);
255     record("repeated query 1 -> 4 is a HIT",
256         "from_cache=1, cost 9", actual,
257         cached != NULL && from_cache == 1 && cached->total_cost == 9);
258
259     /* 14. Cache invalidation after an update ----- */
260     {
261         unsigned long stale0 = cache->stale_evictions;
262         graph_update_road_weight(g, 2, 4, 6); /* 1->4 now 4+6 = 10 */
263         cached = evac_query(g, cache, 1, 4, &from_cache);
264         sprintf(actual, "from_cache=%d, cost %ld, stale_evictions +%lu",
265             from_cache, cached == NULL ? -99 : cached->total_cost,

```

```

266         cache->stale_evictions - stale0);
267     record("cache invalidated after weight update 2-4: 5 -> 6",
268         "from_cache=0, cost 10, stale_evictions +1", actual,
269         cached != NULL && from_cache == 0
270         && cached->total_cost == 10
271         && cache->stale_evictions == stale0 + 1);
272 }
273
274 /* 15. Cyclic graph processed without repetition ----- */
275 {
276     Graph *cyc = graph_create();
277     DijkstraStats stats;
278     int i;
279
280     for (i = 1; i <= 4; i++) {
281         graph_add_intersection(cyc, i);
282     }
283     /* Cycle 1-2-3-1 plus a spur 3-4: bidirectional roads already
284      * create cycles; this adds an explicit triangle. */
285     graph_add_road(cyc, 1, 2, 1);
286     graph_add_road(cyc, 2, 3, 1);
287     graph_add_road(cyc, 3, 1, 1);
288     graph_add_road(cyc, 3, 4, 1);
289     route = dijkstra_shortest_route(cyc, 1, 4, &stats);
290     sprintf(actual, "cost %ld, vertices processed %d (of 4)",
291         route == NULL ? -99 : route->total_cost,
292         stats.vertices_processed);
293     record("cyclic graph: each intersection processed at most once",
294         "cost 2, vertices processed <= 4", actual,
295         route != NULL && route->reachable && route->total_cost == 2
296         && stats.vertices_processed <= 4);
297     route_free(route);
298     graph_free(cyc);
299 }
300
301 /* 16. Isolated vertex ----- */
302 {
303     long reachable = dijkstra_count_reachable(g, 6);
304     route = dijkstra_shortest_route(g, 6, 1, NULL);
305     format_route(route, actual);
306     {
307         char buf[64];
308         sprintf(buf, "; %ld intersection(s) reachable from 6", reachable);
309         strcat(actual, buf);
310     }
311     record("isolated intersection 6 as a source",
312         "unreachable; 1 intersection(s) reachable from 6", actual,
313         route != NULL && !route->reachable && reachable == 1);
314     route_free(route);
315 }
316
317 fprintf(out, "=====\n");
318 fprintf(out, "TOTAL: %d/%d tests passed\n", tests_passed, tests_run);
319
320 route_cache_free(cache);
321 graph_free(g);
322 if (out != stdout) {
323     fclose(out);
324 }
325 printf("test_runner: %d/%d tests passed\n", tests_passed, tests_run);
326 return tests_passed == tests_run ? 0 : 1;
327 }

```

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & ADT Selection CO2	15	Clearly analyzes disaster-routing requirements and selects appropriate ADTs with strong justification	Most requirements identified with appropriate ADTs	Basic understanding of requirements	Limited understanding
Graph, HashMap & HashSet Design-CO2/CO4	20	Efficient graph representation with correct HashMap/HashSet integration	Correct implementation with minor issues	Basic implementation	Inefficient/incomplete design
Min-Heap & Priority Queue Integration-CO4	20	Correct insert, extract-min, heapify/decrease-key operations and efficient integration	Mostly correct implementation	Priority queue works but inefficient	No effective heap implementation
Dijkstra & Route Reconstruction-CO5	20	Correct shortest route, dynamic weights, blocked roads and route reconstruction	Correct with minor issues	Works for simple cases	Major algorithmic errors
Performance, Testing & Complexity Analysis-CO4/CO5	15	Comprehensive scalability, complexity and performance analysis	Good analysis with minor omissions	Basic testing/analysis	Poor or missing analysis
Reflection & SDG Relevance-CO2/CO5	10	Strong technical reflection and clear SDG 9, 11 and 13 connection	Good justification and SDG connection	Generic reflection	Missing/irrelevant reflection

Deliverables

- Pseudocode
- Complete modular C implementation
- Test cases and results
- Complexity analysis
- Dijkstra vs Floyd-Warshall comparison
- Performance analysis
- GitHub repository
- Reflection on design decisions and SDG relevance